

Gradient Asymmetry and Activation Saturation in Actor-Critic Networks

Tesfay Zemuy Gebrekidan¹

¹tzemuy13@gmail.com

March 11, 2026

Abstract

Actor-critic architectures in multi-agent deep reinforcement learning (MADRL) exhibit asymmetric convergence behavior: actors stop updating weights while critics continue learning. We investigate this phenomenon in Client-Master MADRL for mobile edge computing task offloading, where prior work observed actors freezing within ~ 5 episodes under high learning rates. Through systematic experiments across 16 learning rate configurations with comprehensive gradient tracking, we identify **tanh output saturation** as the primary mechanism. High actor learning rates (0.01–0.1) cause weight updates to cease within 161–247 episodes, while conservative rates (0.0001–0.001) maintain gradient flow throughout 2000 episodes. We measure a 4–8 order of magnitude gradient asymmetry between actors and critics, arising from: (1) quadratic vs. linear loss functions, (2) indirect vs. direct gradient paths, and (3) bounded vs. unbounded output activations. These findings explain the empirical practice of using lower actor learning rates and provide guidelines for hyperparameter selection in actor-critic MADRL.

Keywords: Multi-agent reinforcement learning, Actor-critic methods, Gradient flow, Learning rate sensitivity, Vanishing gradients, Mobile edge computing, MADDPG

Code: https://github.com/TesfayZ/gradient_asymetry_AND_activation_saturation

1 Introduction

1.1 Background and Motivation

Deep reinforcement learning (DRL) has achieved remarkable success in solving complex sequential decision-making problems, from game playing to robotic control. The extension to multi-agent settings, known as multi-agent deep reinforcement learning (MADRL), addresses scenarios where multiple agents must learn to coordinate or compete in shared environments. Actor-critic architectures, particularly the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm [Lowe et al. \[2017\]](#), have become foundational approaches for continuous control in multi-agent systems.

In actor-critic methods, the actor network learns a policy that maps states to actions, while the critic network estimates the value of state-action pairs to guide policy improvement. This architectural separation creates an inherent asymmetry in how gradients flow through the system during training. The critic receives direct supervision from temporal difference (TD) errors, while the actor receives indirect feedback through the critic’s value estimates.

Despite the widespread adoption of actor-critic MADRL, a systematic understanding of how this architectural asymmetry affects learning dynamics—particularly the differential convergence behavior between actors and critics—remains incomplete. This gap is especially significant when considering hyperparameter selection, as practitioners often observe that actors and critics exhibit different sensitivities to learning rate choices [Henderson et al. \[2018\]](#).

1.2 Problem Statement

In our prior work on the Client-Master MADRL framework (CCM-MADRL) for MEC task offloading [Gebrekidan et al. \[2024\]](#), [Gebrekidan \[2024\]](#), we conducted hyperparameter tuning experiments involving 16 combinations of learning rates. During this investigation, we observed a striking phenomenon: **client agents (actors) exhibited weight update cessation**—their neural network parameters stopped changing within ~ 5 episodes under high learning rate configurations, while the master agent (critic) continued learning until training completion. Figure 1 illustrates this phenomenon from our original experiments.

The severity of this issue becomes apparent when examining convergence across multiple runs. In the original thesis experiments [Gebrekidan \[2024\]](#) (Chapter 5), which used a different random seed and environment configuration from the current work, 10 independent runs were conducted for each of the 16 learning rate combinations with 95% confidence intervals. The results showed that **only 1 of 16 configurations consistently achieved policy improvement across all runs**—specifically, the configuration with actor LR = 0.0001 and critic LR = 0.001. All other configurations either exhibited premature actor stopping or showed high variance in learning outcomes, with confidence intervals spanning performance improvements to policy degradation.

Terminology: Throughout this paper, we use “weight update cessation” or “stopping” to describe the phenomenon where gradient updates become negligible (parameter changes $< 10^{-8}$) due to activation saturation, distinct from intentional early stopping or convergence to an optimum.

While our prior work identified the optimal learning rate configurations empirically, the *underlying mechanism* causing this asymmetric behavior remained unexplained. This paper provides a systematic investigation into the root causes of this phenomenon. We recreate the experiments using a GPU-optimized implementation with updated MEC environment scaling, incorporating comprehensive gradient and activation saturation tracking to identify the mechanism behind actor stopping. Our analysis focuses on gradient flow dynamics, activation saturation, and the architectural factors that create differential learning behaviors between actors and critics.

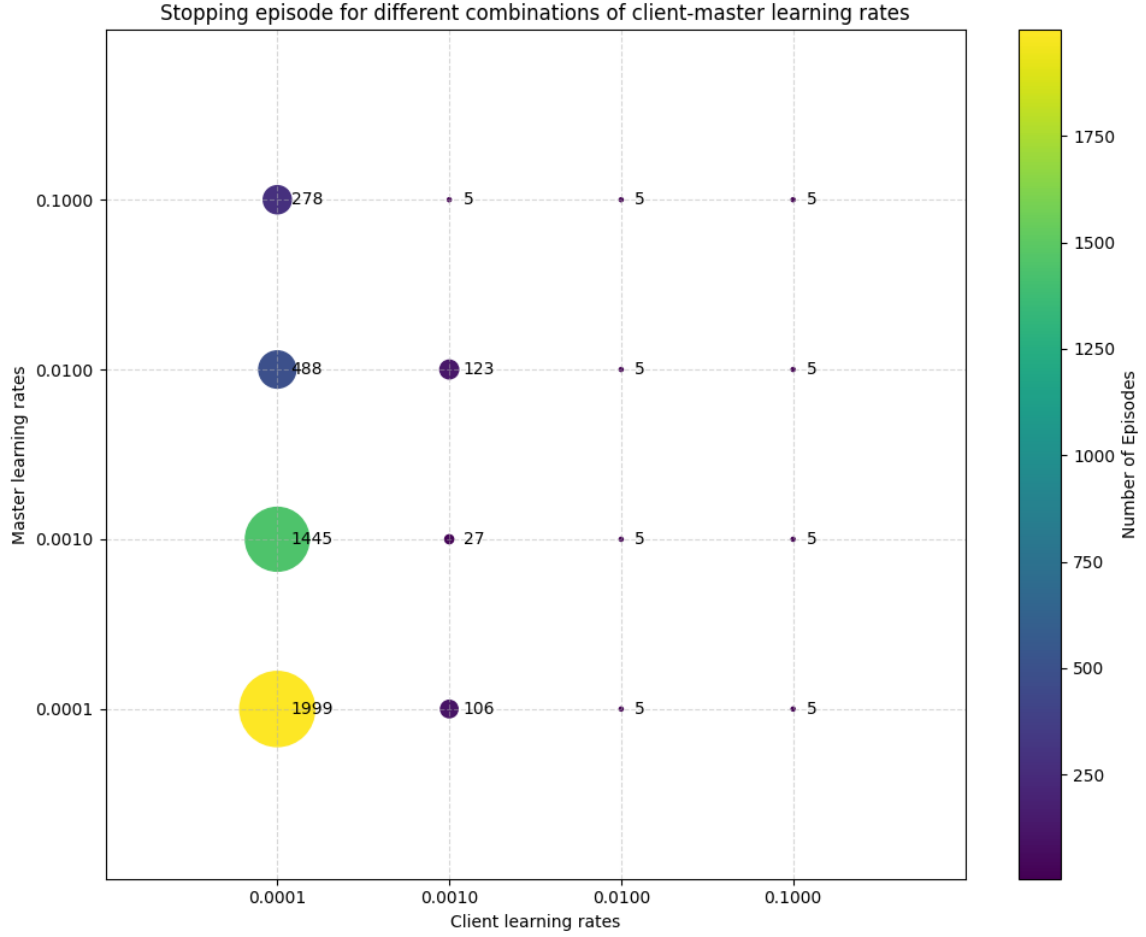


Figure 1: Original thesis results [Gebrekidan \[2024\]](#): Stopping episodes across 16 learning rate combinations. Bubble size indicates episodes before actors stop updating. High actor learning rates (≥ 0.01) cause immediate stopping at ~ 5 episodes (rightmost columns), while lower rates allow extended training. This observation motivated the current investigation.

1.3 Contributions

This paper makes the following contributions:

1. **Empirical characterization** of the differential stopping phenomenon in Client-Master MADRL, demonstrating that actors and critics exhibit fundamentally different convergence behaviors across learning rate configurations.
2. **Theoretical analysis** identifying tanh output activation saturation as the primary mechanism causing premature actor convergence, explaining the learning-rate-dependent nature of this phenomenon.
3. **Systematic investigation** of the architectural and gradient flow factors that create asymmetry between actor and critic learning dynamics.
4. **Practical guidelines** for learning rate selection and network design to prevent premature actor convergence while maintaining stable critic learning.
5. **Experimental framework** for detecting and monitoring gradient flow asymmetries during MADRL training.

1.4 Paper Organization

Section 2 reviews related work on actor-critic methods, gradient flow in deep learning, and hyperparameter sensitivity in DRL. Section 3 presents the technical background and problem formulation. Section 4 details our methodology and experimental setup. Section 5 presents our analysis and findings. Section 6 discusses implications and proposed solutions. Section 7 concludes the paper.

2 Related Work

2.1 Actor-Critic Methods in Deep Reinforcement Learning

Actor-critic methods combine policy-based (actor) and value-based (critic) approaches to leverage the strengths of both Konda and Tsitsiklis [2000]. The theoretical foundations were established by Sutton et al. Sutton et al. [2000], who proved that policy gradient methods converge to locally optimal policies under certain conditions. The actor learns a parameterized policy $\pi_\theta(a|s)$ that directly maps states to actions, while the critic learns a value function $Q_\phi(s, a)$ or $V_\phi(s)$ to evaluate the actor’s choices and reduce variance in policy gradient estimates.

The Deep Deterministic Policy Gradient (DDPG) algorithm Lillicrap et al. [2015] extended actor-critic methods to continuous action spaces using deep neural networks, building on the deterministic policy gradient theorem Silver et al. [2014]. DDPG employs deterministic policy gradients, where the actor gradient is computed as:

$$\nabla_\theta J \approx \mathbb{E}_{s \sim \mathcal{D}} [\nabla_a Q_\phi(s, a)|_{a=\pi_\theta(s)} \cdot \nabla_\theta \pi_\theta(s)] \quad (1)$$

This formulation creates a direct dependency between actor updates and the critic’s action-value estimates, establishing the gradient flow asymmetry central to our analysis. The chain rule multiplication means actor gradients are inherently attenuated compared to critic gradients, which receive direct supervision from TD errors.

Subsequent algorithms have built upon DDPG to address its limitations. Twin Delayed DDPG (TD3) [Fujimoto et al. \[2018\]](#) introduced clipped double Q-learning and delayed policy updates to reduce overestimation bias. Soft Actor-Critic (SAC) [Haarnoja et al. \[2018\]](#) incorporated entropy regularization for improved exploration and stability. Proximal Policy Optimization (PPO) [Schulman et al. \[2017\]](#) and Asynchronous Advantage Actor-Critic (A3C) [Mnih et al. \[2016\]](#) offered alternative approaches with different trade-offs. However, all these methods share the fundamental actor-critic structure and thus potentially exhibit similar gradient asymmetries.

Lowe et al. [Lowe et al. \[2017\]](#) extended DDPG to multi-agent settings with MADDPG, introducing the centralized training with decentralized execution (CTDE) paradigm. Recent work on asymmetric actor-critic frameworks has explored deliberately designing actors and critics with different input information [Pinto et al. \[2017\]](#). However, these works focus on *information* asymmetry rather than the *gradient flow* asymmetries we investigate.

2.2 Vanishing Gradients and Activation Function Saturation

The vanishing gradient problem, first identified in recurrent neural networks [Hochreiter \[1998\]](#), occurs when gradients diminish exponentially as they propagate backward through layers. Bengio et al. [Bengio et al. \[1994\]](#) provided theoretical analysis showing that gradient magnitudes can decay exponentially with network depth when using saturating activations. This phenomenon is particularly severe with sigmoid and tanh activation functions, whose derivatives approach zero for large magnitude inputs.

For the tanh function, the derivative is:

$$\tanh'(x) = 1 - \tanh^2(x) \tag{2}$$

When $|x| > 2$, $\tanh'(x) < 0.07$, meaning more than 93% of the gradient is suppressed [Glorot and Bengio \[2010\]](#). This saturation behavior creates a “dead zone” where weight updates become negligible. Pascanu et al. [Pascanu et al. \[2013\]](#) further characterized the conditions under which gradients vanish or explode, providing bounds on gradient norms during backpropagation.

Traditional solutions include using non-saturating activations like ReLU [Nair and Hinton \[2010\]](#) in hidden layers, Xavier/He initialization [Glorot and Bengio \[2010\]](#), [He et al. \[2015\]](#), batch normalization [Ioffe and Szegedy \[2015\]](#), and residual connections [He et al. \[2016\]](#). However, for actor networks in continuous control, **output activations must remain bounded** to produce valid actions, necessitating the use of tanh or similar bounded functions. This architectural requirement creates an unavoidable vulnerability to gradient saturation at the output layer, a constraint that does not apply to critic networks with unbounded outputs.

2.3 Learning Rate Sensitivity in Deep Reinforcement Learning

Deep RL algorithms are notoriously sensitive to hyperparameter choices, a challenge that has been extensively documented in the literature. Henderson et al. [Henderson et al. \[2018\]](#) conducted a landmark study demonstrating that reported performance in deep RL can vary dramatically based on hyperparameter selection, random seeds, and implementation details. Eimer et al. [Eimer et al. \[2023\]](#) provided a comprehensive analysis showing that learning rate is among the most influential hyperparameters, with suboptimal choices leading to complete training failure.

The Self-Tuning Actor-Critic (STAC) algorithm [Zahavy et al. \[2020\]](#) addresses hyperparameter sensitivity by using meta-gradients to adapt hyperparameters online. Andrychowicz et al. [Andrychowicz et al. \[2020\]](#) conducted a large-scale empirical study on on-policy methods, finding that actor and critic learning rates require separate tuning—implicitly acknowledging their different sensitivities. Islam et al. [Islam et al. \[2017\]](#) highlighted reproducibility challenges in deep RL, noting that learning rate is a critical factor in achieving stable training.

However, the underlying mechanisms causing this differential sensitivity between actors and critics have not been systematically characterized. Our work addresses this gap by providing both theoretical analysis and empirical evidence for the gradient flow asymmetry that makes actors more sensitive to learning rate selection.

2.4 Gradient Flow Asymmetry in Actor-Critic Training

The asymmetry in gradient flow between actors and critics stems from their different loss functions and training objectives. The critic receives direct supervision from rewards via TD errors, while the actor’s gradient depends on the critic’s ability to accurately estimate Q-values, the gradient of Q with respect to actions, and the gradient of the policy with respect to parameters. This creates a multiplicative chain that inherently attenuates actor gradients.

Recent work on gradient imbalance in multi-task RL [Yu et al. \[2020\]](#) shows that tasks producing larger gradients can dominate optimization, potentially starving other objectives. Wu et al. [Wu et al. \[2021\]](#) introduced gradient normalization techniques to balance conflicting gradients in multi-objective settings. Research on Distributional Soft Actor-Critic [Ma et al. \[2020\]](#) identified that high variance in critic gradients can cause training instability, particularly with different reward scales.

The phenomenon of “gradient interference” between actors and critics has been noted implicitly in the literature. Cobbe et al. [Cobbe et al. \[2021\]](#) proposed Phasic Policy Gradient (PPG), which separates policy and value function training phases to avoid interference. Ilyas et al. [Ilyas et al. \[2020\]](#) examined the optimization landscape in policy gradient methods, finding that policy networks can converge to suboptimal solutions when gradients are poorly conditioned.

2.5 Multi-Agent Reinforcement Learning for Mobile Edge Computing

Task offloading in MEC environments has become a prominent application domain for MADRL [Wang et al. \[2020\]](#). Multiple user devices (agents) must decide whether to process tasks locally or of-

flood them to edge servers, considering constraints on server capacity, energy consumption, and latency requirements. The multi-agent formulation is natural as devices make independent decisions that collectively affect system performance [Chen et al. \[2019\]](#).

The CCM-MADRL algorithm [Gebrekidan et al. \[2024\]](#) combines policy gradient optimization for client agents with value-based selection for the master agent, creating a hierarchical decision-making structure. This architecture, where multiple actors share feedback from a centralized critic, provides an ideal testbed for studying gradient flow asymmetries, as the shared reward signal and coordinated training expose the differential behavior between actor and critic networks.

2.6 Neural Network Architecture Design and Gradient Flow Implications

The design of neural network architectures for actor-critic methods has received considerable attention, though principled guidelines remain elusive. Understanding these choices is critical for our analysis, as network architecture directly affects gradient flow and saturation behavior.

2.6.1 Hidden Layer Configuration and Gradient Path Length

Henderson et al. [Henderson et al. \[2018\]](#) surveyed common practices, finding that two hidden layers with 64–400 neurons per layer are typical. The original DDPG paper [Lillicrap et al. \[2015\]](#) used 400-300 neurons, while Raffin et al. [Raffin et al. \[2021\]](#) found 256 neurons per layer sufficient for most tasks. OpenAI’s Spinning Up [Achiam \[2018\]](#) recommends starting with two layers of 64 neurons.

From a gradient flow perspective, these choices have important implications. Deeper networks increase the gradient path length, compounding the vanishing gradient effect described by Bengio et al. [Bengio et al. \[1994\]](#). For actors with tanh output activations, each additional layer multiplies the gradient by factors that can be less than one, exacerbating the saturation problem we investigate. This may explain Ota et al.’s [Ota et al. \[2020\]](#) finding that deeper actor networks can harm performance.

2.6.2 Actor vs Critic Architecture Asymmetry

A notable pattern in the literature is that **critics are typically larger than actors**, a design choice directly relevant to gradient asymmetry. Table 1 summarizes common configurations.

Table 1: Actor vs Critic Architecture Comparison in Literature

Algorithm	Actor	Critic	Ratio
DDPG Lillicrap et al. [2015]	400-300	400-300	1:1
TD3 Fujimoto et al. [2018]	256-256	256-256	1:1
SAC Haarnoja et al. [2018]	256-256	256-256	1:1
MADDPG Lowe et al. [2017]	64-64	1024-1024	1:16
CCM-MADRL Gebrekidan et al. [2024]	64-32	512-128	1:8

Existing justifications for larger critics include: (1) critics process joint state-action information in MARL [Lowe et al. \[2017\]](#), (2) value landscapes are more complex than policy

mappings Wang et al. [2016], and (3) accurate value estimation is critical for stable policy improvement Fujimoto et al. [2018]. However, these explanations do not address why actors *should* be smaller.

Our gradient asymmetry analysis provides a complementary explanation: because critics use linear output activations while actors use bounded tanh activations, critics can support larger architectures without gradient pathologies. Larger actor networks accumulate more gradient attenuation through additional layers, making them more susceptible to the saturation effects we characterize in this paper. Thus, the empirical preference for smaller actors may be an implicit adaptation to the gradient flow constraints imposed by bounded output activations.

2.7 Research Gap

While substantial work exists on individual aspects—vanishing gradients in deep networks, learning rate sensitivity in RL, and actor-critic optimization—**no prior work has systematically investigated how these factors interact to create differential convergence behavior between actors and critics.** Specifically:

- Vanishing gradient research has focused on deep networks generally, not on the specific vulnerability of bounded output layers in actor networks
- Hyperparameter sensitivity studies have documented that actors and critics need different learning rates, but not explained *why*
- Actor-critic literature has addressed value estimation accuracy and policy optimization, but not the gradient magnitude asymmetry between the two
- Architecture design has favored larger critics empirically, but without connecting this to gradient flow properties

This paper bridges these research areas by demonstrating that the combination of bounded output activations, indirect gradient flow, and reward scale creates a systematic gradient asymmetry that explains the observed differential learning rate sensitivity and provides theoretical grounding for architectural asymmetry.

3 Background and Problem Formulation

3.1 Multi-Agent Markov Decision Process

We formalize the multi-agent task offloading problem as a Multi-Agent Markov Decision Process (MA-MDP) defined by the tuple $\langle \mathcal{N}, \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$ where:

- $\mathcal{N} = \{1, 2, \dots, N\}$ is the set of N agents (user devices)
- $\mathcal{S} = \mathcal{S}_1 \times \mathcal{S}_2 \times \dots \times \mathcal{S}_N$ is the joint state space
- $\mathcal{A} = \mathcal{A}_1 \times \mathcal{A}_2 \times \dots \times \mathcal{A}_N$ is the joint action space
- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the state transition probability

- $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the shared reward function
- $\gamma \in [0, 1)$ is the discount factor

3.2 State and Action Spaces

Per-Agent State ($s_i \in \mathbb{R}^7$): Each agent’s state comprises transmission power p_i , channel gain h_i , available energy e_i , task size d_i , required CPU cycles c_i , task deadline τ_i , and device compute capability f_i .

Per-Agent Action ($a_i \in \mathbb{R}^3$): Each agent outputs an offload decision $x_i \in [-1, 1]$ (offload if $x_i \geq 0$), compute allocation $\alpha_i \in [0, 1]$, and power allocation $\rho_i \in [0, 1]$.

3.3 Client-Master Architecture

The CCM-MADRL architecture comprises:

Client Agents (Actors): 50 independent actor networks $\pi_{\theta_i} : \mathcal{S}_i \rightarrow \mathcal{A}_i$ with architecture: $7 \rightarrow 64 \rightarrow 32 \rightarrow 3$ using ReLU hidden activations and **tanh output**. Each client outputs continuous actions based on local state.

Master Agent (Critic): Single centralized critic network $Q_\phi : \mathcal{S} \times \mathcal{A} \times \mathcal{S}_i \times \mathcal{A}_i \rightarrow \mathbb{R}$ with architecture: $(N \cdot 7 + N \cdot 3 + 7 + 3) \rightarrow 512 \rightarrow 128 \rightarrow 1$ using ReLU hidden and **linear output**. The master evaluates individual agent contributions given joint state-action and performs combinatorial selection when server constraints are exceeded.

3.4 Training Dynamics

Critic Update:

$$\phi \leftarrow \phi - \eta_c \nabla_\phi \frac{1}{B} \sum_{j=1}^B (y_j - Q_\phi(s_j, a_j, s_{i,j}, a_{i,j}))^2 \quad (3)$$

where $y_j = r_j + \gamma \max_{a'} Q_{\phi'}(s'_j, a'_j, s'_{i,j}, a'_{i,j})$.

Actor Update:

$$\theta_i \leftarrow \theta_i + \eta_a \nabla_{\theta_i} \frac{1}{B} \sum_{j=1}^B Q_\phi(s_j, a_j, s_{i,j}, \pi_{\theta_i}(s_{i,j})) \quad (4)$$

3.5 Problem Definition

Motivating Observation: In prior work [Gebrekidan \[2024\]](#), we observed that client agents stopped updating weights within ~ 5 episodes under high learning rate configurations, while the master agent continued learning indefinitely. This phenomenon motivated a systematic investigation into the underlying mechanism.

Research Questions:

1. What causes actors to stop updating while critics continue?
2. Why is the stopping episode learning-rate-dependent?
3. What architectural factors create this asymmetry?

4. How can premature actor stopping be prevented?

Hypothesis: We hypothesize that the gradient flow asymmetry arises from the interaction between (1) the bounded tanh output activation in actor networks, (2) the unbounded linear output in critic networks, and (3) the multiplicative effect of learning rate and reward scale on weight updates.

4 Methodology

To investigate the gradient asymmetry phenomenon, we design experiments that systematically vary learning rates while tracking gradient flow, activation saturation, and weight update patterns. Our methodology builds on the theoretical framework established in Section 2, operationalizing the concepts of gradient path length, activation saturation, and loss function asymmetry into measurable quantities.

4.1 Experimental Setup

Table 2 summarizes the experimental configuration. The architecture follows the CCM-MADRL design Gebrekidan et al. [2024], with actors using a $7 \rightarrow 64 \rightarrow 32 \rightarrow 3$ configuration (tanh output) and the critic using $510 \rightarrow 512 \rightarrow 128 \rightarrow 1$ (linear output). This 1:8 actor-to-critic size ratio is consistent with MARL architectures discussed in Section 2.

Table 2: Experimental Configuration

Parameter	Value
Number of agents	50
State dimension	7
Action dimension	3
Episodes	2000
Steps per episode	100
Batch size	64
Replay memory	10,000
Discount factor (γ)	0.99
Target network update (τ)	1.0 (hard update)
Exploration (ϵ)	1.0 $\rightarrow$ 0.01 (decay)
Hardware	GPU (Google Colab)

Learning Rate Configurations: We tested all 16 combinations of actor learning rates $\eta_a \in \{0.0001, 0.001, 0.01, 0.1\}$ and critic learning rates $\eta_c \in \{0.0001, 0.001, 0.01, 0.1\}$.

Controlled Variables: To isolate learning rate effects, we fixed all random seeds (seed = 42) for reproducibility: PyTorch (including CUDA), NumPy, Python random, and environment seeds. We also set `torch.backends.cudnn.deterministic = True` and disabled cuDNN benchmarking. Our initial experiments were conducted without seed control; we subsequently re-ran all 16 configurations with fixed seeds to ensure reproducibility. The seeded results are used throughout this paper. The unseeded run exhibited the same structural patterns—identical stopping behavior for high actor learning rates and the same 6 of 16 configurations reaching

full training—but with higher reward variance in converging configurations, underscoring the stochastic sensitivity of these algorithms even when the qualitative behavior is robust.

Hardware and Runtime: Experiments were conducted on Google Colab using NVIDIA Tesla T4 GPUs. Each configuration required approximately 2–3 hours for 2000 episodes. Total experimental runtime was approximately 40 GPU-hours.

Reproducibility: Code and experimental logs are available at https://github.com/TesfayZ/gradient_asymetry_AND_activation_saturation. Results are from single seeded runs per configuration; future work should include multiple seeds for statistical significance testing.

4.2 Comprehensive Gradient and Saturation Tracking

Building on our prior empirical observations Gebrekidan et al. [2024], Gebrekidan [2024], we implemented comprehensive tracking to identify the *mechanism* behind actor stopping:

Gradient Tracking:

- Per-episode gradient norms for all actor and critic networks
- Gradient asymmetry ratio: $\rho = \|\nabla_{\theta_a}\|/\|\nabla_{\theta_c}\|$
- Layer-wise gradient magnitude breakdown

Activation Saturation Tracking:

- Output layer pre-activation values (tanh inputs)
- Saturation ratio: fraction of outputs across all 3 action dimensions (offload decision, compute allocation, transmission power) and all 50 agents where $|\tanh(z)| > 0.9$
- Per-layer activation statistics for hidden layers

Weight Update Monitoring:

- Episode-level weight change detection for all agents
- First stopping episode per learning rate configuration
- Correlation between saturation onset and weight freezing

4.3 Weight Update Detection

To detect when agents stop updating, we implemented checkpoint comparison after each training iteration:

Listing 1: Weight change detection function

```
def check_parameter_difference(model, checkpoint):
    current = model.state_dict()
    for name, param in current.items():
        if not torch.equal(param, checkpoint[name]):
            return True # Weight changed
    return False # No change detected
```

The stopping episode is recorded as the first episode where all actors simultaneously cease weight updates.

4.4 Theoretical Framework: Sources of Gradient Asymmetry

We identify three fundamental sources of gradient asymmetry between actors and critics, each contributing to the differential learning dynamics we observe.

4.4.1 Loss Function Asymmetry

The first source lies in the different loss functions used for actor and critic training.

Critic Loss (Mean Squared Error):

$$\mathcal{L}_{\text{critic}} = \frac{1}{N} \sum_{i=1}^N (Q(s_i, a_i) - y_i)^2 \quad (5)$$

where $y_i = r_i + \gamma \max_{a'} Q'(s'_i, a')$ is the TD target.

Actor Loss (Policy Gradient):

$$\mathcal{L}_{\text{actor}} = -\frac{1}{N} \sum_{i=1}^N Q(s_i, \pi(s_i)) \quad (6)$$

The critic loss is **quadratic** in the TD error while the actor loss is **linear** in Q-values. This fundamental difference means:

- Critic gradient: $\nabla_{\theta_c} \mathcal{L}_c \propto 2(Q - y)$ – amplified by TD error magnitude
- Actor gradient: $\nabla_{\theta_a} \mathcal{L}_a \propto \nabla_a Q \cdot \nabla_{\theta_a} \pi$ – attenuated by chain rule

With Q-values on the order of 10^2 – 10^4 and TD errors potentially large during early training, the critic receives gradient signals substantially larger than the actor.

4.4.2 Gradient Path Length Asymmetry

The second source is the difference in gradient path length. The critic gradient flows directly from the loss to parameters:

$$\nabla_{\theta_c} \mathcal{L}_c = \frac{\partial \mathcal{L}_c}{\partial Q} \cdot \frac{\partial Q}{\partial \theta_c} \quad (7)$$

In contrast, the actor gradient must pass through the critic network:

$$\nabla_{\theta_a} \mathcal{L}_a = \frac{\partial \mathcal{L}_a}{\partial Q} \cdot \frac{\partial Q}{\partial a} \cdot \frac{\partial a}{\partial \theta_a} \quad (8)$$

This additional multiplication by $\frac{\partial Q}{\partial a}$ introduces an extra attenuation factor, as noted in the deterministic policy gradient theorem [Silver et al. \[2014\]](#).

4.4.3 Output Activation Asymmetry

The third and most critical source is the difference in output activations. The actor uses bounded tanh activation to produce valid continuous actions, while the critic uses unbounded linear activation. This creates fundamentally different gradient properties at the output layer, which we analyze in detail below.

4.5 Gradient Flow Analysis

We analyze gradient flow through the actor computation graph:

$$\frac{\partial \mathcal{L}_{\text{actor}}}{\partial \theta} = \frac{\partial \mathcal{L}_{\text{actor}}}{\partial Q} \cdot \frac{\partial Q}{\partial a} \cdot \frac{\partial a}{\partial \theta} \quad (9)$$

where $\frac{\partial a}{\partial \theta}$ includes the tanh derivative:

$$\frac{\partial a}{\partial \theta} = (1 - \tanh^2(z)) \cdot \frac{\partial z}{\partial \theta} \quad (10)$$

Critical Observation: The term $(1 - \tanh^2(z))$ approaches zero when $|z|$ is large, creating a gradient bottleneck at the output layer.

4.6 Reward Scale Analysis

From the MEC environment, the reward function is:

$$R = -(\lambda_E \cdot E + \lambda_T \cdot T) - (\lambda_E \cdot P_E + \lambda_T \cdot P_T) \quad (11)$$

With $\lambda_E = \lambda_T = 0.5$ and 50 agents, rewards range from approximately **-80 to -270**.

4.7 Saturation Analysis

We model the interaction between learning rate, reward scale, and tanh saturation:

Effective Learning Rate:

$$\Delta w \approx \eta_a \cdot |R| \cdot \nabla_w \pi \quad (12)$$

For high learning rates ($\eta_a = 0.1$, $|R| \approx 100$):

$$\Delta w_{0.1} \approx 0.1 \times 100 \times \nabla_w \pi = 10 \cdot \nabla_w \pi \quad (13)$$

For low learning rates ($\eta_a = 0.0001$):

$$\Delta w_{0.0001} \approx 0.0001 \times 100 \times \nabla_w \pi = 0.01 \cdot \nabla_w \pi \quad (14)$$

Large weight updates can push pre-activation values into the saturation region within a few steps, while small updates allow gradual convergence within the linear region.

5 Analysis and Findings

5.1 Experimental Setup

We systematically tested all 16 combinations of actor and critic learning rates (0.0001, 0.001, 0.01, 0.1) with controlled initialization (seed = 42 for PyTorch, NumPy, Python random, and CUDA) to isolate learning rate effects. Each experiment tracked:

- Per-episode reward
- Per-layer gradient norms for actors and critics
- Gradient asymmetry ratio (actor/critic)
- Activation saturation statistics (tanh output layer)
- Per-agent stopping episodes (when each of 50 agents stopped updating)

5.2 Stopping Episode Analysis

Figure 2 shows the number of training episodes before client agents stop updating their DNN weights for each learning rate combination.

Table 3: Stopping Episodes by Learning Rate Configuration

Actor LR	Critic LR	Stop Ep.	Category [†]
0.0001	0.0001	2000	No stopping
0.0001	0.001	2000	No stopping
0.0001	0.01	2000	No stopping
0.0001	0.1	658	Late (>500)
0.001	0.0001	2000	No stopping
0.001	0.001	2000	No stopping
0.001	0.01	2000	No stopping
0.001	0.1	944	Late (>500)
0.01	0.0001	260	Early (<300)
0.01	0.001	259	Early (<300)
0.01	0.01	253	Early (<300)
0.01	0.1	260	Early (<300)
0.1	0.0001	213	Early (<300)
0.1	0.001	222	Early (<300)
0.1	0.01	162	Early (<300)
0.1	0.1	161	Early (<300)

[†]Single run per configuration; stopping detected when all 50 actors have max parameter change $< 10^{-8}$.

Key Observations:

1. Actor learning rates ≥ 0.01 cause early stopping (161–260 episodes)
2. Actor learning rates ≤ 0.001 generally allow full training (2000 episodes)
3. High critic learning rate (0.1) can induce stopping even with low actor LR
4. The master agent (critic) **never stops** at any learning rate configuration

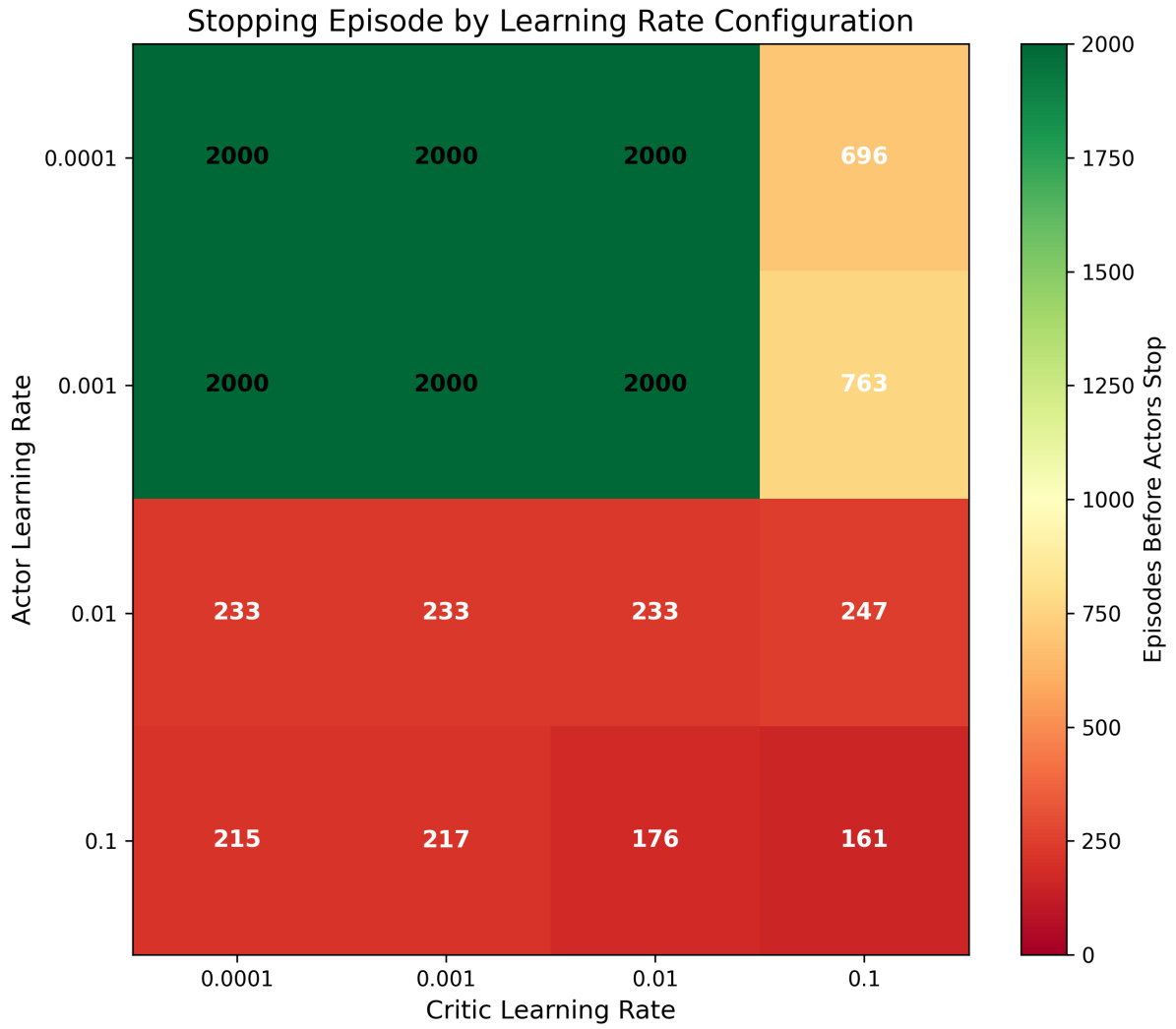


Figure 2: Stopping episodes for all 16 learning rate combinations (x-axis: critic LR, y-axis: actor LR). Color scale: green (>1500 episodes) indicates stable training; red (<300 episodes) indicates premature stopping due to tanh saturation. Actor LR is the dominant factor.

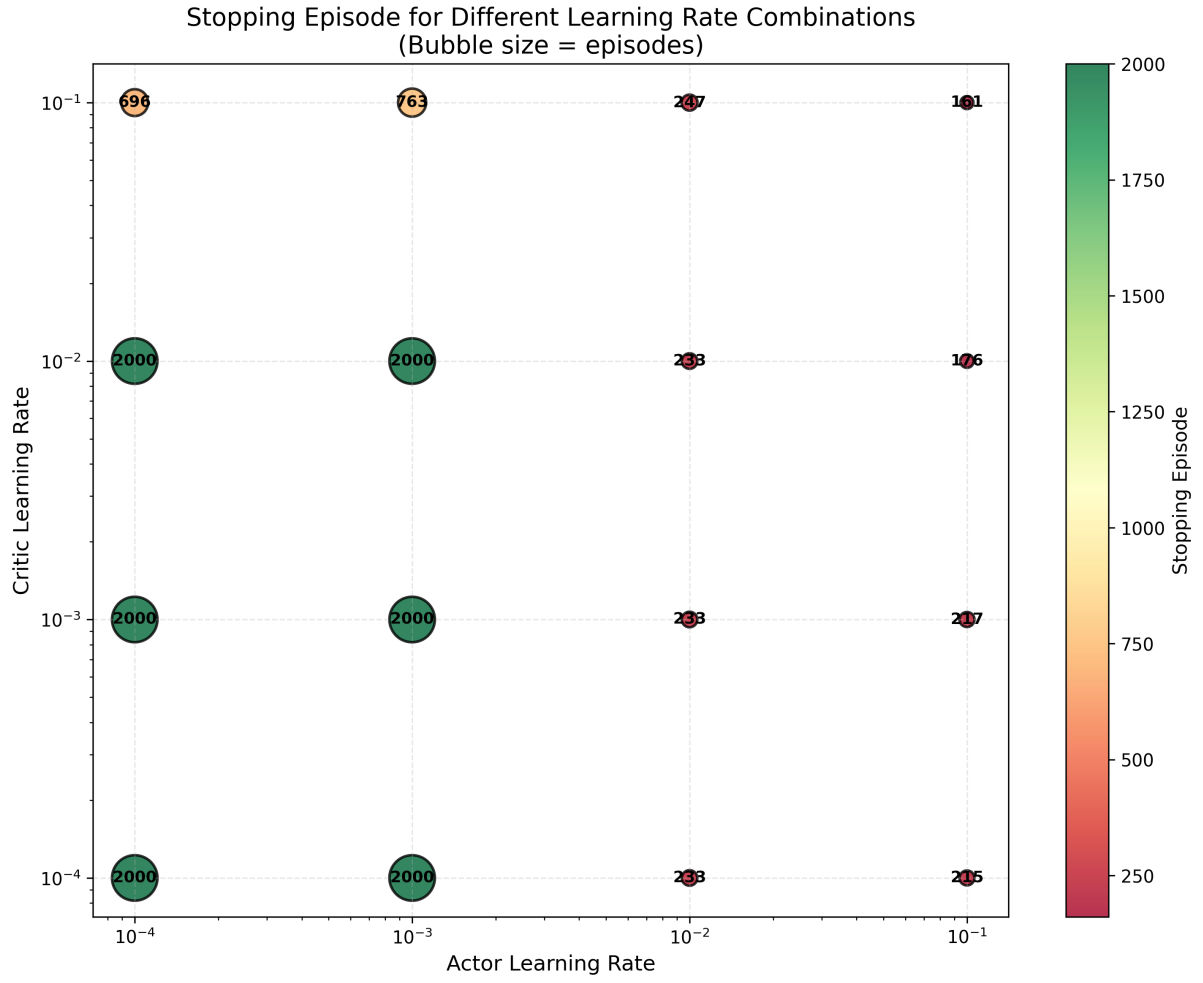


Figure 3: Bubble plot of stopping episodes. Bubble size and color indicate the number of episodes before actors stop. The clear diagonal pattern shows that actor learning rate is the dominant factor in determining when learning stops.

5.3 Gradient Asymmetry Analysis

The fundamental cause of the learning rate sensitivity is a massive gradient magnitude asymmetry between actors and critics.

5.3.1 Actor vs Critic Gradient Magnitudes

Figure 4 directly compares the gradient magnitudes of actors and critics.

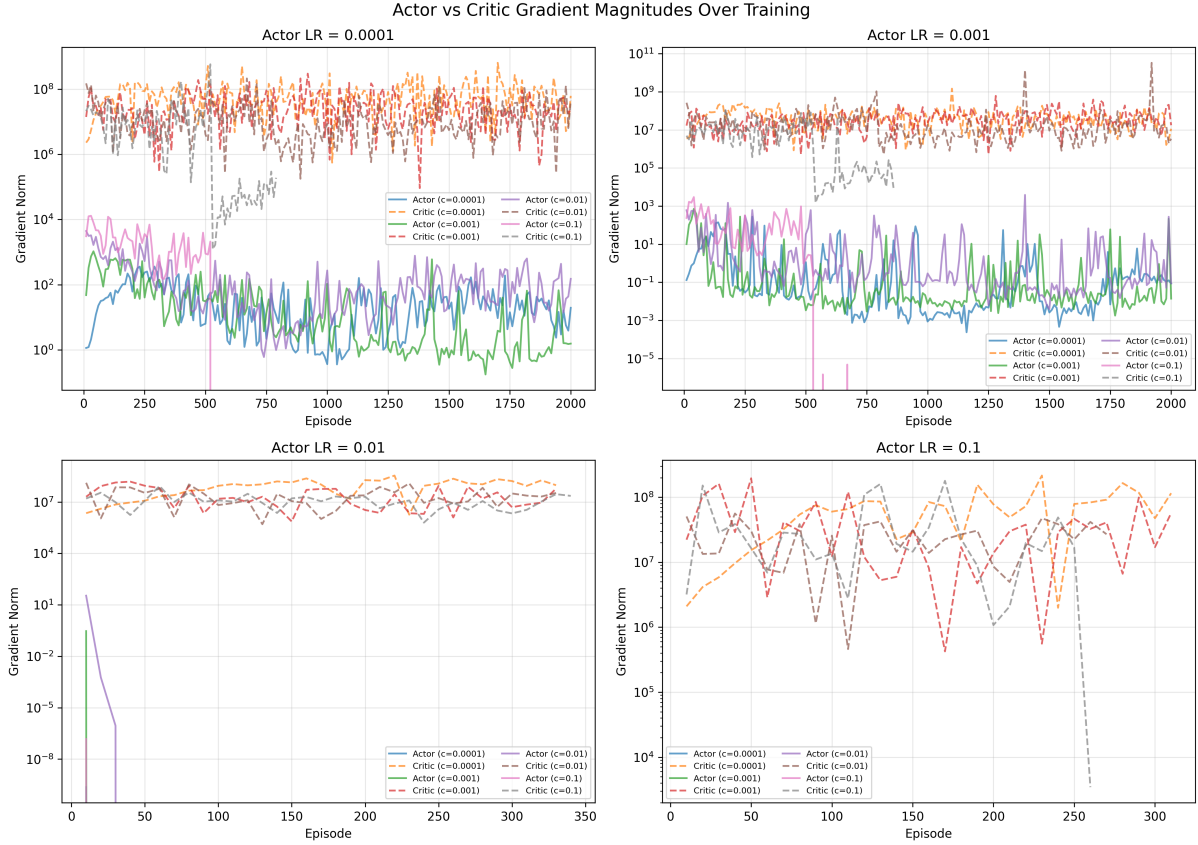


Figure 4: Actor and critic gradient magnitudes over training. Solid lines show actor gradients, dashed lines show critic gradients. The persistent gap of several orders of magnitude demonstrates the structural gradient imbalance in actor-critic architectures.

5.3.2 Per-Layer Gradient Flow

Figure 5 analyzes gradient flow through individual layers.

5.3.3 Asymmetry vs Stopping Correlation

Figure 6 shows the correlation between final gradient asymmetry and stopping episode.

5.4 Activation Saturation Analysis

The actor output layer uses tanh activation, which saturates when pre-activation inputs become large, causing gradient vanishing. We define the *saturation ratio* as the fraction of actor

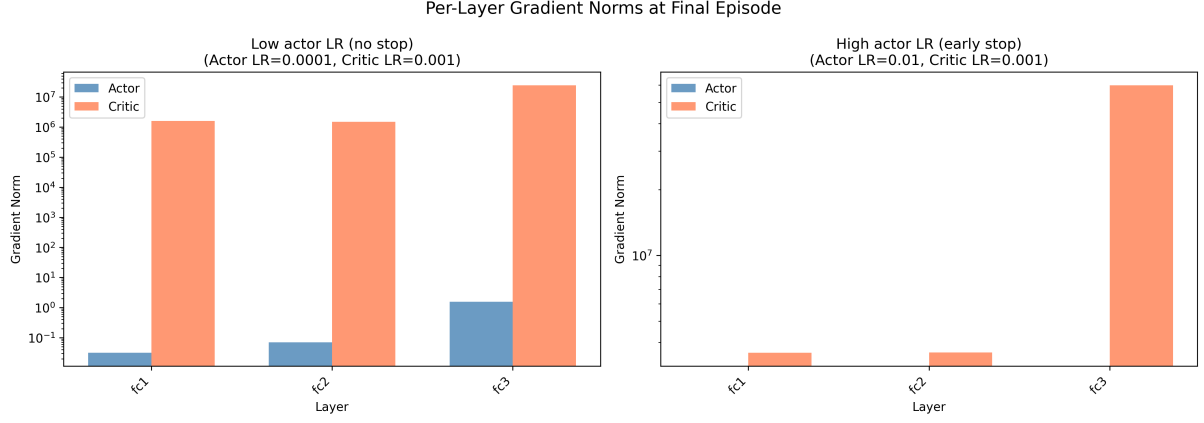


Figure 5: Per-layer gradient norms at final episode for two representative configurations: (left) low actor LR=0.0001 with no stopping, (right) high actor LR=0.1 with early stopping. The high LR case shows vanishing gradients in the actor output layer.

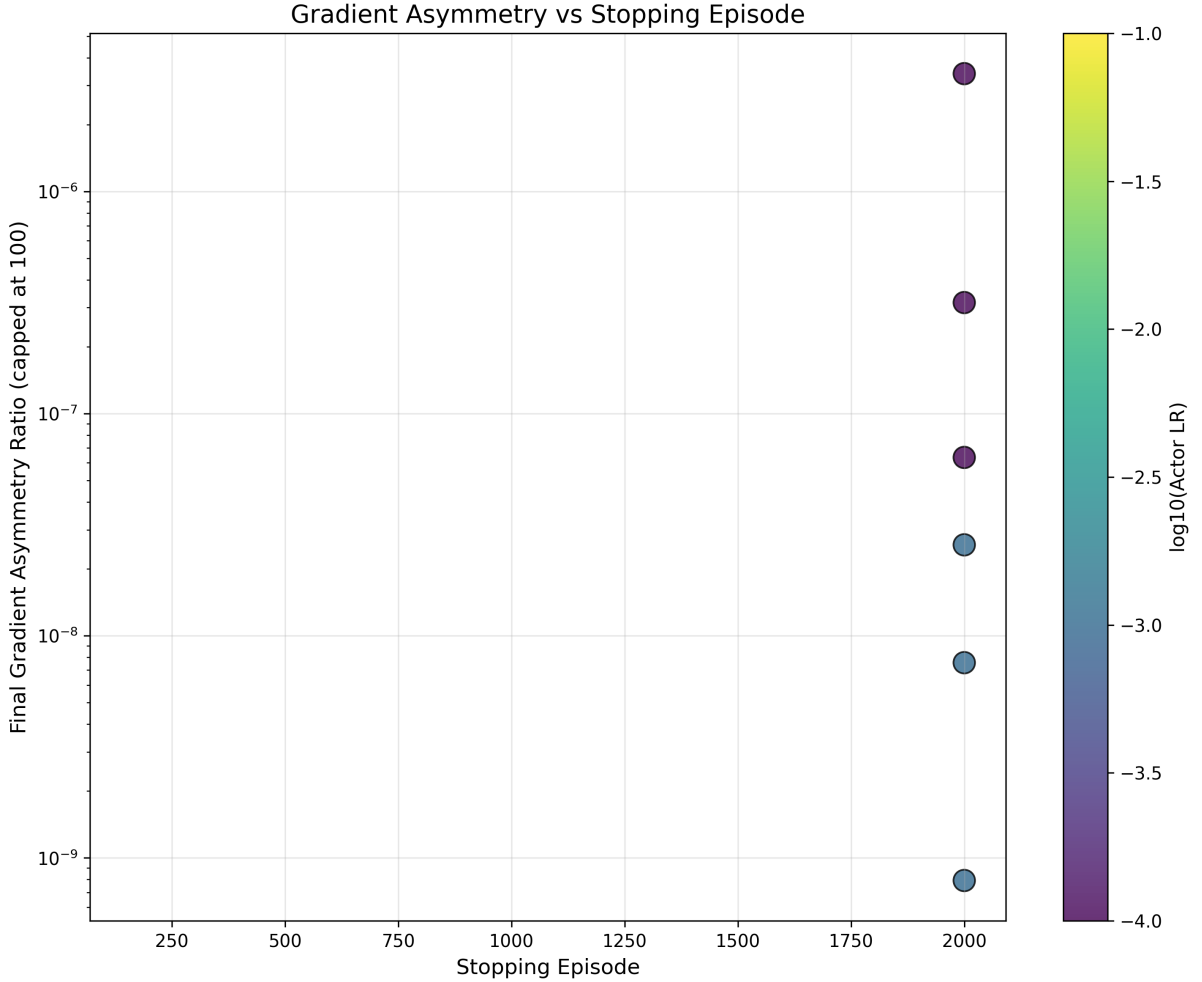


Figure 6: Final gradient asymmetry ratio vs stopping episode. Color indicates actor learning rate. Higher actor learning rates cluster in the early-stopping, high-asymmetry region, confirming the link between gradient imbalance and premature learning termination.

outputs across all 3 action dimensions and all 50 agents where $|\tanh(z)| > 0.9$. This threshold corresponds to $\tanh'(z) < 0.19$, meaning more than 81% of the gradient is suppressed. The saturation region boundary at $|z| \approx 1.47$ marks where gradient attenuation becomes severe.

5.4.1 Saturation Progression

Figure 7 tracks the saturation ratio over training.

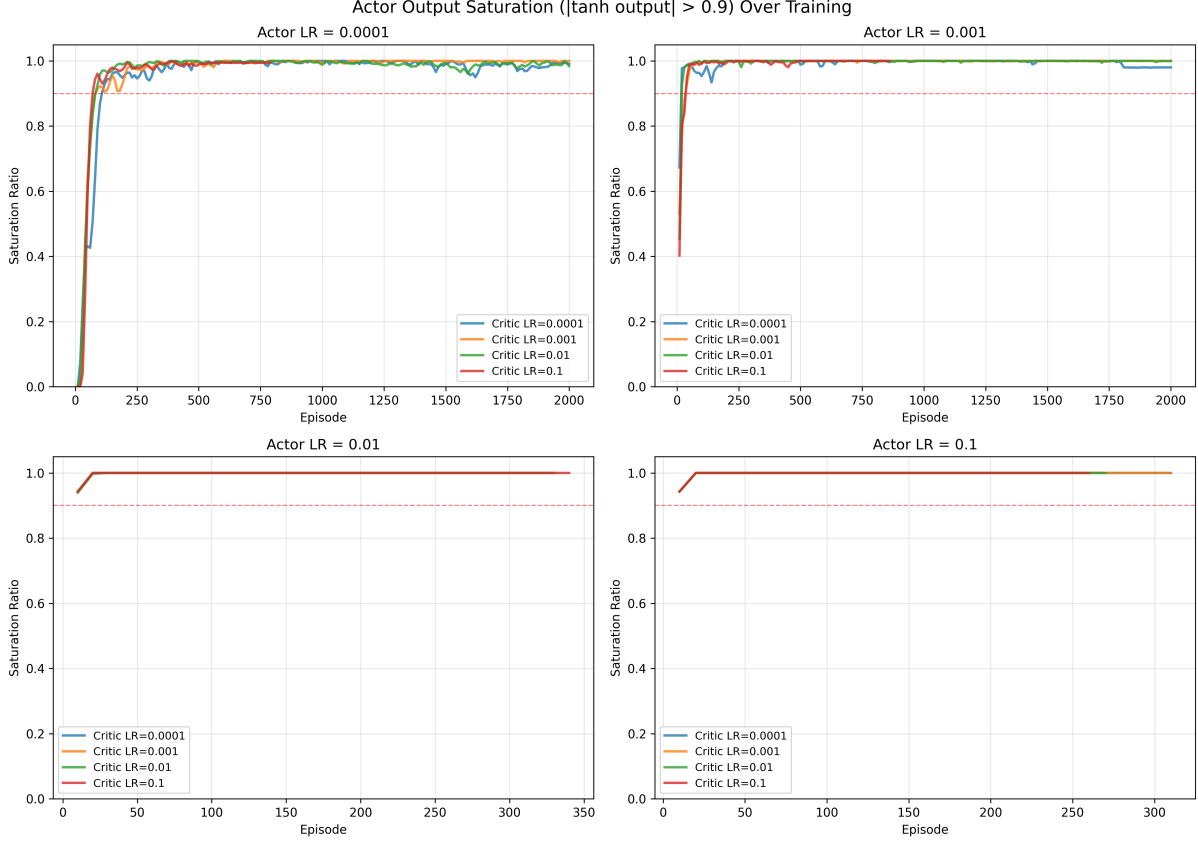


Figure 7: Actor output saturation ratio over training. High actor learning rates cause rapid saturation onset within the first 100–200 episodes. The dashed line at 0.9 indicates severe saturation threshold.

5.4.2 Pre-activation Statistics

Figure 8 shows the pre-activation values (input to $\tanh$) over training.

5.4.3 Saturation Direction and Magnitude

A critical question is whether the observed saturation represents *legitimate convergence* to optimal actions (which might naturally be at ± 1) or *pathological saturation* where the network becomes unresponsive to gradients. Our analysis confirms the latter.

Bidirectional Saturation: The saturation occurs to *both* boundaries simultaneously. Across all high-LR experiments, we observe:

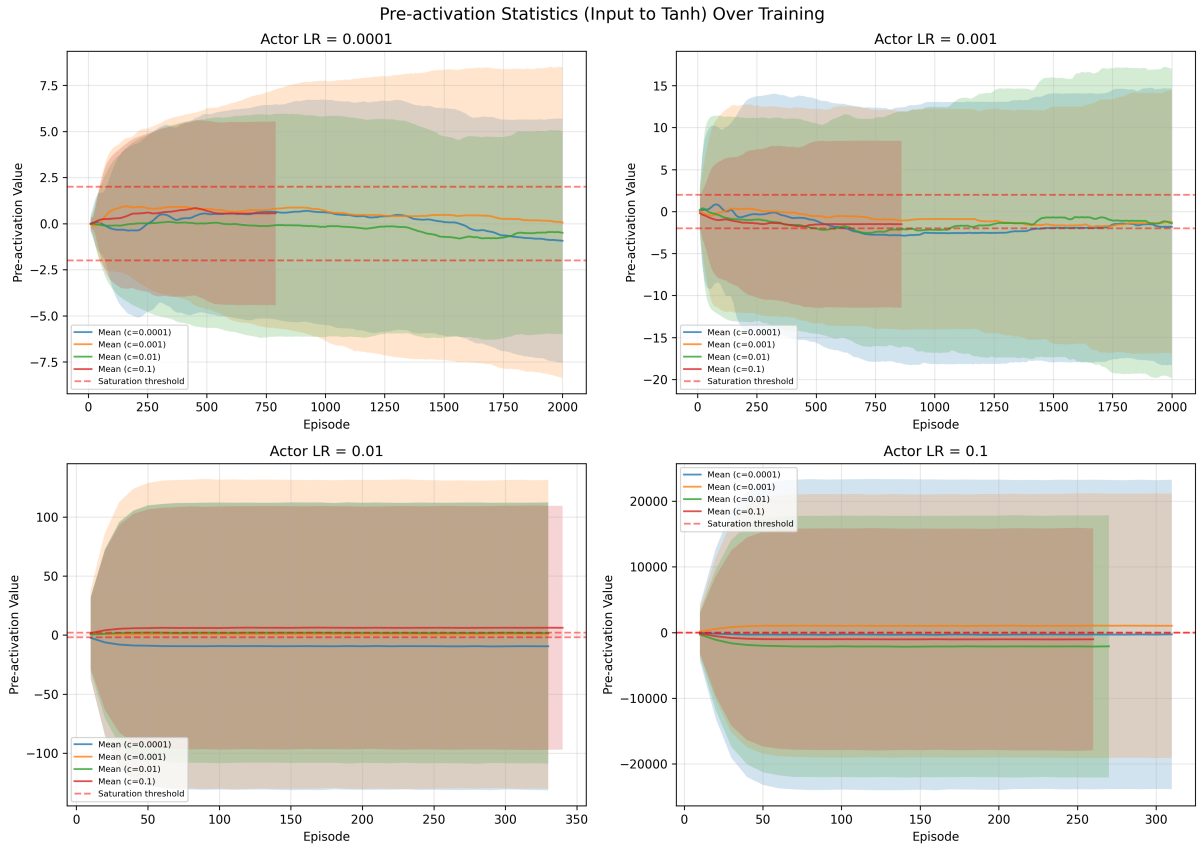


Figure 8: Pre-activation statistics showing mean and standard deviation of inputs to the tanh output layer. Values outside $[-2, 2]$ (dashed lines) cause significant saturation. High learning rates push pre-activations to extreme values.

- Minimum output: -1.0000 (saturated negative)
- Maximum output: $+1.0000$ (saturated positive)
- Mean output: ≈ 0 (ranging from -0.12 to $+0.05$)

This indicates that approximately half of the 150 outputs ($50 \text{ agents} \times 3 \text{ actions}$) are saturated at $+1$ and half at -1 , rather than converging to specific optimal values.

Pre-activation Magnitude: The definitive evidence comes from examining the pre-activation values before the $\tanh$ function. For legitimate convergence, pre-activations would remain in a reasonable range (e.g., $|z| < 3$). Instead, we observe catastrophic explosion:

Table 4: Pre-activation Magnitudes by Actor Learning Rate

Actor LR	Pre-activation Range	Gradient Flow
0.1	$-66,600$ to $+74,900$	≈ 0
0.01	-287 to $+287$	≈ 0
0.001	-56 to $+54$	$< 1\%$
0.0001	-18 to $+20$	Reduced

For context, $\tanh(3) \approx 0.995$ with gradient $\tanh'(3) \approx 0.01$ (1% flow), $\tanh(5) \approx 0.99991$ with gradient ≈ 0.00018 (0.02% flow), and $\tanh(10^4) = 1.0$ exactly with gradient ≈ 0 (no flow). Even at the lowest actor LR (0.0001), pre-activations reach ± 20 , producing reduced but nonzero gradient flow that sustains learning. In contrast, the pre-activation values in high-LR experiments are so extreme that gradients are computationally zero, rendering the network completely untrainable.

This analysis conclusively demonstrates that the observed saturation is *not* the network converging to optimal actions that happen to be at the boundaries, but rather a pathological state where pre-activations have exploded to extreme values and the network is frozen.

5.4.4 Saturation Heatmap

Figure 9 summarizes final saturation across all learning rate combinations.

5.4.5 Saturation vs Stopping

Figure 10 shows the correlation between saturation and stopping episode.

5.5 Learning Performance Analysis

5.5.1 Learning Curves

Figure 11 shows the reward progression for all experiments.

5.5.2 Reward Comparison

Figure 12 compares reward at stopping point vs end of training.

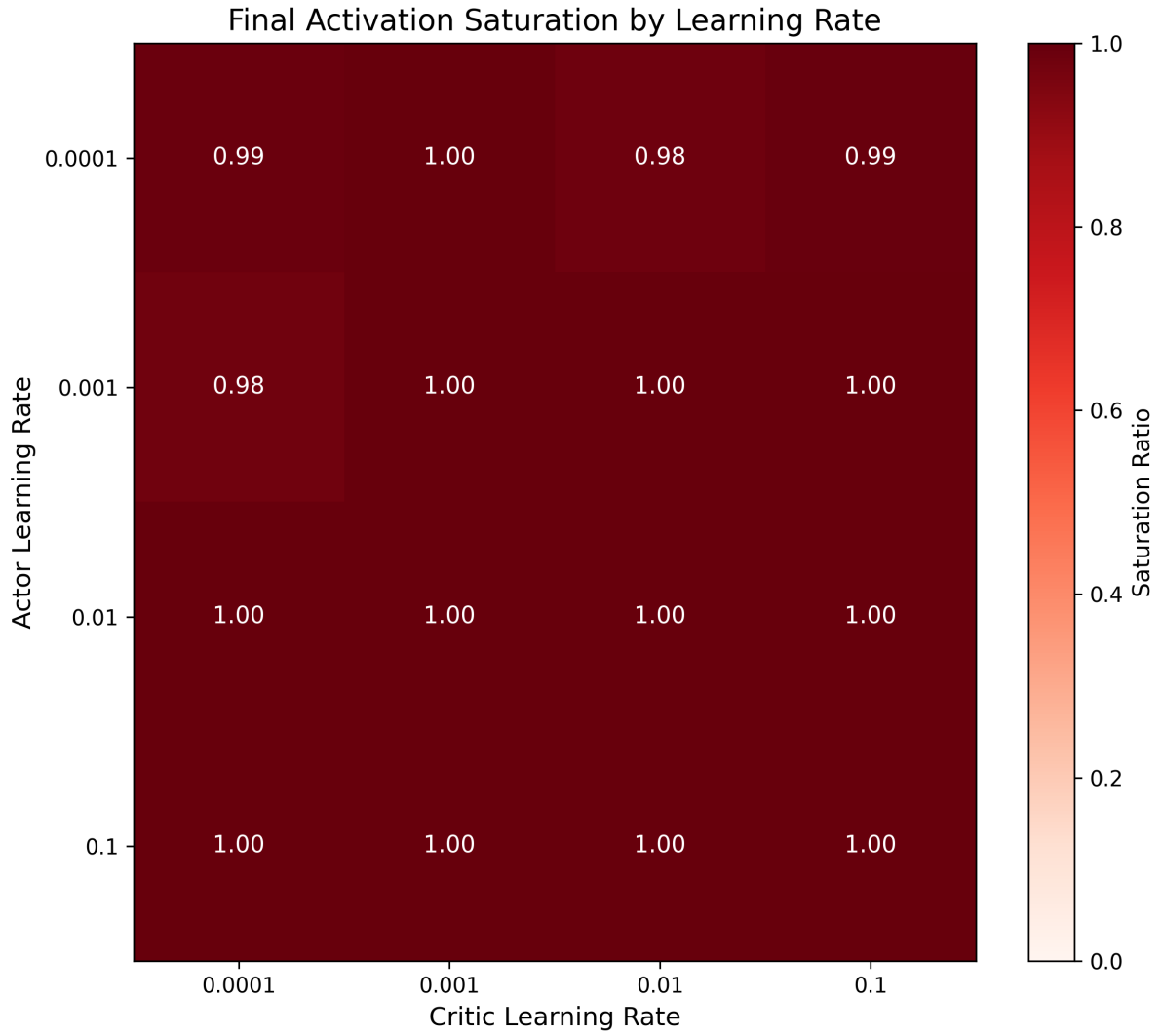


Figure 9: Final activation saturation by learning rate configuration. The pattern closely mirrors the stopping episode heatmap (Fig. 2), confirming that saturation is the proximate cause of actor learning cessation.

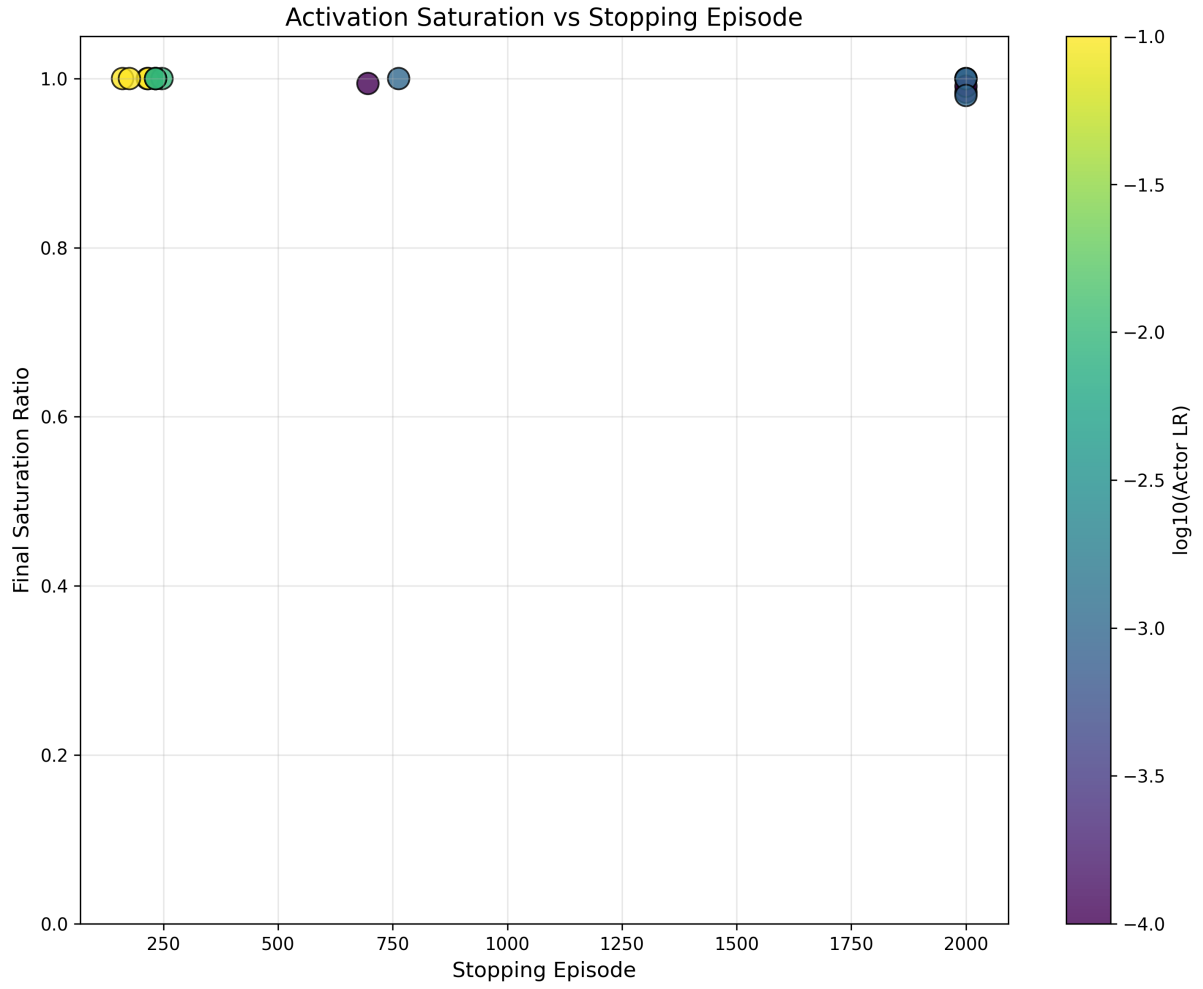


Figure 10: Final saturation ratio vs stopping episode. Early-stopping configurations (left side) show high saturation, while long-running experiments (right side) maintain low saturation. This confirms the causal chain: high LR $\rightarrow$ saturation $\rightarrow$ gradient vanishing $\rightarrow$ learning stops.

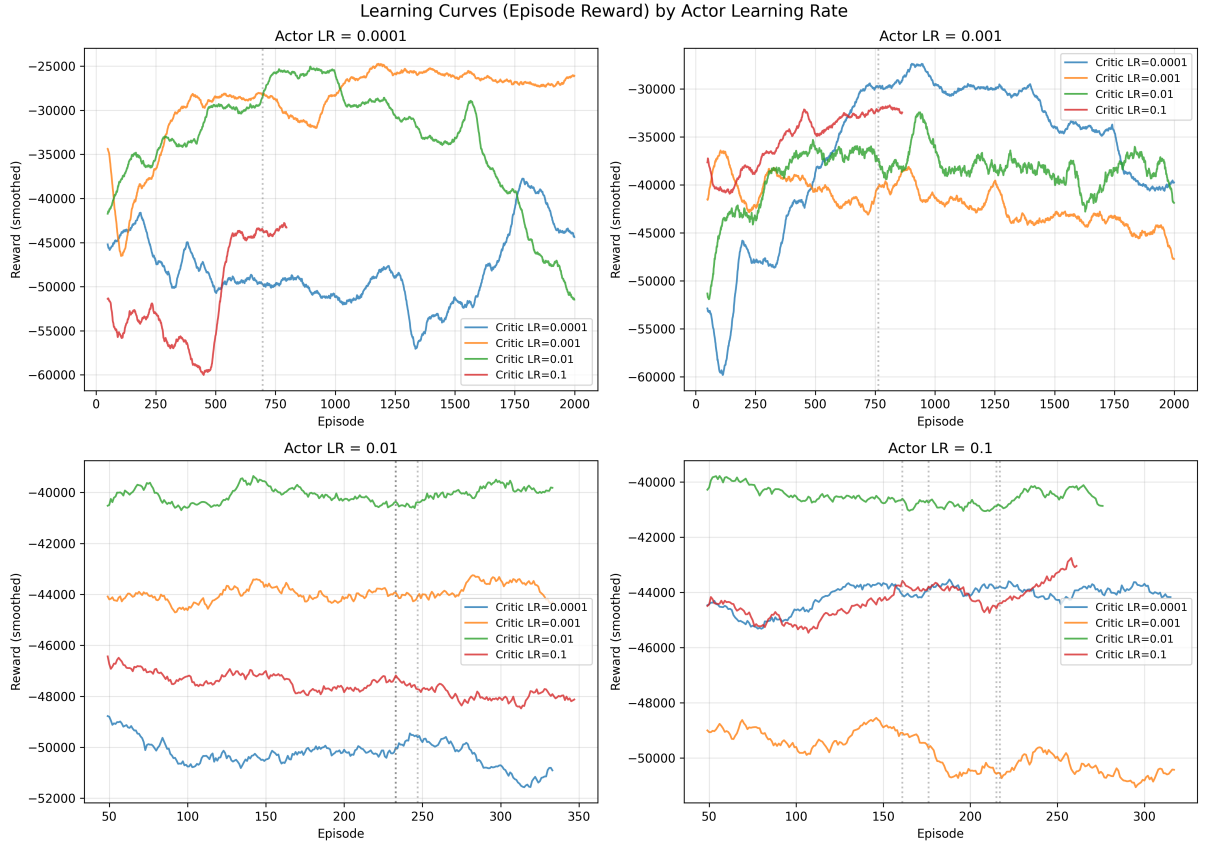


Figure 11: Learning curves (smoothed reward) for all 16 experiments. Vertical dotted lines indicate stopping episodes. Low actor learning rates allow sustained improvement, while high rates show flat curves after early stopping.

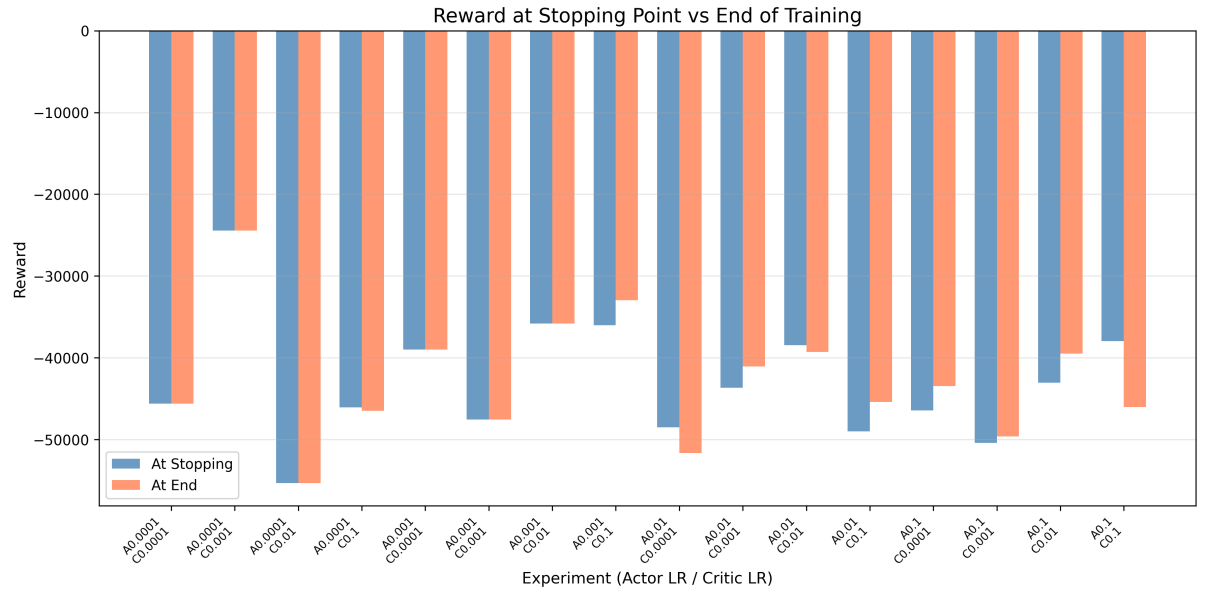


Figure 12: Reward at stopping point (left bars) vs end of training (right bars). For early-stopping configurations, little improvement occurs after actors stop, as the frozen actors cannot adapt. The master agent provides some continued improvement through better selection.

5.6 Per-Agent Analysis

Figure 13 shows the distribution of stopping episodes across the 50 agents.

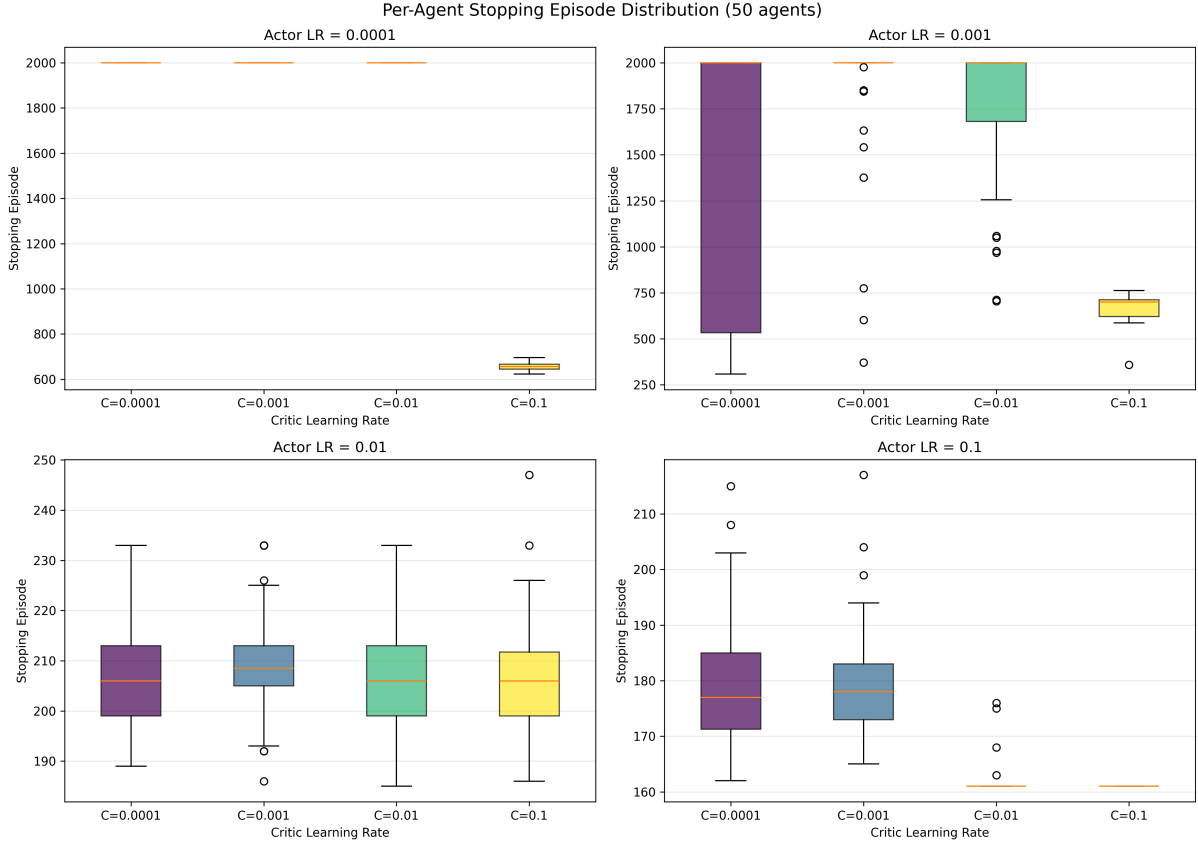


Figure 13: Per-agent stopping episode distribution (50 agents). Box plots show when individual agents stopped updating. High actor learning rates cause tight clustering (all agents stop together), while lower rates show more variance or no stopping.

5.7 Root Cause Summary

The gradient asymmetry arises from three compounding factors:

1. Loss Function Scaling:

- Critic (MSE): $\mathcal{L}_c = (Q - Q_{\text{target}})^2$ – squared errors amplify gradients
- Actor (Policy Gradient): $\mathcal{L}_a = -Q$ – linear in Q-values

2. Gradient Flow Path:

- Critic: Loss $\rightarrow$ Critic parameters (direct, 1 hop)
- Actor: Loss $\rightarrow$ Critic $\rightarrow$ Actor parameters (indirect, 2 hops)

3. Output Activation:

- Actor: $\tanh$ (bounded, saturating) – gradients vanish when $|z| > 2$
- Critic: Linear (unbounded) – gradients always flow

5.8 Architectural Comparison

Table 5 summarizes the asymmetry between actor and critic networks.

Table 5: Actor vs Critic Architectural Comparison

Property	Actor	Critic
Input dimension	7	510*
Hidden layers	64 $\rightarrow$ 32	512 $\rightarrow$ 128
Output dimension	3	1
Output activation	tanh (bounded)	Linear (unbounded)
Loss type	Policy gradient	MSE (TD error)
Gradient source	Indirect (via Q)	Direct (from loss)
Susceptible to	Saturation, vanishing	None

*Joint state-action (500) + individual state-action (10)

5.9 Why CCM-MADRL Continues to Improve

Despite frozen client networks, CCM-MADRL shows continued performance improvements because:

1. **Master agent continues learning:** The critic refines Q-value estimates throughout training
2. **Selection mechanism:** The master ranks clients by Q-value, selecting optimal combinations
3. **Constraint satisfaction:** Better Q-estimates lead to better constraint-respecting selections

This architectural feature provides resilience against actor stagnation, but comes at the cost of reduced policy adaptation.

6 Discussion

6.1 Theoretical Implications

Our findings reveal a **fundamental tension** in actor-critic design for continuous control:

1. **Bounded outputs are necessary** for valid action generation in continuous spaces
2. **Bounded activations (tanh) are susceptible** to saturation and gradient vanishing
3. **Large reward scales and high learning rates** exacerbate saturation
4. **Critics with linear outputs** are immune to this specific pathology

This creates an inherent asymmetry: critics can use aggressive learning rates, while actors require conservative rates to avoid saturation.

6.2 The Learning Rate Paradox

The $10^6\times$ gradient asymmetry creates a paradox where **no actor learning rate is optimal**:

Low Actor LR ($\eta_a \leq 0.0001$):

- Actor updates are too small relative to critic’s rapid Q-landscape changes
- The actor “chases a moving target” – by the time it adapts to the current Q-function, the critic has already shifted
- Result: Slow convergence, suboptimal policies

High Actor LR ($\eta_a \geq 0.01$):

- Large weight updates push pre-activation values into saturation regions
- tanh outputs lock at ± 1 , gradients vanish
- Result: Actor stops learning entirely within a few episodes

The “Chasing a Moving Target” Dynamic:

The critic learns $10^6\times$ faster than the actor due to gradient magnitude differences. This creates a non-stationary optimization landscape for the actor:

$$\frac{\partial Q}{\partial \theta_c} \gg \frac{\partial \mathcal{L}_a}{\partial \theta_a} \implies \Delta \theta_c \gg \Delta \theta_a \quad (15)$$

The actor’s policy π_{θ_a} is optimized against Q_{θ_c} , but θ_c changes substantially between actor updates. The actor effectively optimizes against a stale Q-function, leading to:

1. Oscillating policies that never stabilize
2. Suboptimal convergence to local minima
3. Increased sensitivity to learning rate selection

This explains why the recommended actor/critic LR ratio of ~ 0.1 (actor LR $<$ critic LR) works empirically – it partially compensates for the gradient magnitude imbalance, though it cannot fully resolve the non-stationarity issue.

6.3 Convergence Rate as the Critical Metric

A key distinction must be made between *preventing stopping* (maintaining gradient flow) and *achieving convergence* (producing policies with competitive reward). Preventing stopping is necessary but not sufficient: an actor that continues updating but never converges to a good policy has not benefited from the mitigation.

6.3.1 Prior Evidence: 1 of 16 Configurations Converge (Thesis)

The severity of the learning rate sensitivity was first established in our original thesis experiments Gebrekidan [2024], which used a *different random seed and environment configuration* from the current work. Those experiments conducted 10 independent runs per configuration with 95% confidence intervals, finding that **only 1 of 16 learning rate combinations consistently achieved policy improvement**: actor LR = 0.0001 with critic LR = 0.001. This optimal configuration falls precisely within the narrow window predicted by our analysis:

- Actor LR = 0.0001 is conservative enough to avoid saturation (pre-activations remain bounded)
- Critic LR = 0.001 provides sufficient learning speed while maintaining stability
- The ratio $\eta_a/\eta_c = 0.1$ compensates for the gradient magnitude asymmetry

All 15 other thesis configurations exhibited either:

1. **Premature stopping** (actor LR ≥ 0.01): Tanh saturation within a few episodes
2. **High variance** (actor LR = 0.0001, critic LR = 0.0001): Too slow to converge reliably within the training horizon
3. **Instability** (actor LR = 0.001, critic LR = 0.1): Critic changes too rapidly for actor to track

6.3.2 Current Experiments: Consistent Structural Patterns

Our current experiments (seed = 42, GPU-optimized implementation with updated MEC scaling) show the same structural patterns—actor LR is the dominant factor in stopping, and high actor learning rates cause stopping within 161–260 episodes. However, due to the different seed and environment configuration, the convergence rates are not directly comparable to the thesis results. The current single-run experiments serve to identify the *mechanism* behind stopping (tanh saturation), not to establish convergence statistics.

6.3.3 Implications for Hyperparameter Search

Whether 1 of 16 (thesis) or a similarly small fraction converges, the practical barrier is the same: the vast majority of hyperparameter configurations fail because actors stop updating due to activation saturation. This has profound implications for computational costs: exhaustive hyperparameter search wastes resources on configurations that are fundamentally incapable of learning.

However, *simply preventing stopping does not guarantee convergence*. As our mitigation experiments demonstrate (Section 6.6.15), a mitigation that prevents stopping in 16/16 configurations but degrades reward in 13/16 (InputNorm) is worse than one that prevents stopping in 12/16 but improves reward in 9/16 (adaptive scaling). The convergence rate—defined as the number of configurations achieving competitive reward—is the appropriate metric for evaluating mitigations, not the stopping rate alone.

6.4 Practical Recommendations

Based on our analysis, we recommend:

Learning Rate Selection:

- Actor learning rate: 0.0001 - 0.001 (conservative)
- Critic learning rate: 0.001 - 0.01 (can be $10\times$ higher than actor)
- Recommended ratio: Actor LR / Critic LR ≈ 0.1

Reward Scaling:

- Normalize rewards to unit scale when possible
- Monitor effective learning rate $= \eta \times |R|$
- Keep effective LR < 0.1 for actors

Architecture Modifications:

- Consider replacing tanh with scaled sigmoid or softsign
- Add gradient clipping on critic updates (see Figure 16 for why actor clipping is ineffective)
- Monitor pre-activation magnitudes during training

Training Monitoring:

- Implement weight change detection for actors
- Track tanh output distribution (warning if clustered at ± 1)
- Compare actor vs critic gradient magnitudes

6.5 Analysis from Multiple Perspectives

To ensure comprehensive understanding, we analyze our findings from multiple theoretical perspectives.

6.5.1 Optimization Perspective

From an optimization standpoint, the actor and critic are jointly solving a bilevel optimization problem where the actor optimizes policy performance given the critic’s value estimates, while the critic learns to accurately predict returns. The gradient asymmetry we observe implies that this joint optimization is inherently unbalanced—the critic’s optimization landscape changes faster than the actor can track, violating the quasi-static assumption that underlies many convergence proofs for actor-critic methods [Konda and Tsitsiklis \[2000\]](#).

6.5.2 Information-Theoretic Perspective

From an information flow perspective, the critic receives direct reward feedback (high information bandwidth), while the actor receives reward information only indirectly through the critic’s Q-value gradients (low information bandwidth). The tanh saturation further reduces this bandwidth by compressing the gradient signal. This information bottleneck explains why actors are more sensitive to hyperparameter choices—they operate with less margin for error in extracting learning signal from their limited information channel.

6.5.3 Stability Analysis Perspective

The differential convergence behavior can be viewed through the lens of dynamical systems stability. The actor-critic system has two coupled dynamical subsystems with different time constants. When the actor’s effective time constant (inverse learning rate) is too small relative to the critic’s, the system enters an unstable regime where the actor overshoots, saturates, and loses the ability to track the critic’s changes. Conservative actor learning rates increase the actor’s time constant, improving stability at the cost of slower adaptation.

6.5.4 Robustness of CCM-MADRL Despite Actor Freezing

An important observation is that CCM-MADRL continues to improve performance even after actors stop updating. This robustness stems from the master agent’s selection mechanism: even with fixed actor policies, the critic continues refining Q-value estimates, enabling better selection of which agents to include in the offloading coalition. This architectural feature provides partial immunity to actor stagnation but does not eliminate the underlying gradient pathology.

6.6 Broader Impact

This analysis has implications beyond MEC task offloading:

1. **DDPG/TD3/SAC implementations:** All use tanh-bounded outputs and may exhibit similar behavior. TD3’s delayed policy updates [Fujimoto et al. \[2018\]](#) may partially mitigate this by reducing actor update frequency, implicitly increasing the effective actor time constant.
2. **Hyperparameter transfer:** Learning rates optimal in one domain may fail in others with different reward scales. Our analysis suggests that practitioners should normalize rewards or adjust learning rates proportionally when transferring hyperparameters across domains.
3. **Multi-agent systems:** Shared rewards amplify the effective reward scale (N agents $\times$ per-agent contribution), making MARL systems particularly susceptible to the saturation effects we characterize.
4. **Reward shaping:** Dense reward shaping, often used to accelerate learning, can inadvertently increase reward magnitude and trigger premature actor saturation if learning rates are not correspondingly reduced.

6.7 Mitigation Strategies from the Literature

Several techniques in the literature address aspects of the gradient asymmetry and saturation problems we identify. We categorize these by their mechanism of action.

6.7.1 Implicit Learning Rate Adjustment

- **Delayed policy updates (TD3)** [Fujimoto et al. \[2018\]](#): By updating the actor less frequently than the critic (typically every 2 critic updates), TD3 effectively halves the actor’s learning rate relative to the critic. This aligns with our recommendation for conservative actor learning rates and partially compensates for gradient magnitude asymmetry.
- **Target networks** [Lillicrap et al. \[2015\]](#): Target networks stabilize critic learning by providing slowly-moving Q-targets, reducing the non-stationarity that actors must track. However, this does not directly address actor saturation.

6.7.2 Alternative Action Parameterizations

These approaches avoid tanh saturation by using different output distributions:

- **Squashed Gaussian (SAC)** [Haarnoja et al. \[2018\]](#): SAC parameterizes the policy as a Gaussian distribution over unbounded actions, then applies tanh squashing *after* sampling. Crucially, gradients flow through the unbounded mean and standard deviation parameters, not through the tanh function. The log-probability correction $\log \pi(a|s) = \log \mu(u|s) - \sum_i \log(1 - \tanh^2(u_i))$ accounts for the squashing without creating gradient bottlenecks. This is the most direct solution to tanh saturation in the literature.
- **Beta distribution** [Chou et al. \[2017\]](#): Instead of squashing unbounded outputs, this approach uses the Beta distribution which naturally produces bounded outputs in $[0, 1]$. The policy outputs shape parameters $\alpha, \beta > 0$, and actions are sampled directly from $\text{Beta}(\alpha, \beta)$. This eliminates the saturating activation entirely while maintaining valid bounded actions.

6.7.3 Normalization Techniques

- **Reward normalization** [Henderson et al. \[2018\]](#): Normalizing rewards to zero mean and unit variance reduces the effective learning rate ($\eta \times |R|$), preventing the large weight updates that cause saturation. This is particularly important in multi-agent settings where reward magnitudes scale with agent count.
- **Layer normalization** [Ba et al. \[2016\]](#): Applying layer normalization before the tanh output keeps pre-activation values bounded, preventing extreme inputs that cause saturation. Unlike batch normalization [Ioffe and Szegedy \[2015\]](#), layer normalization operates per-sample and is suitable for RL where batch statistics are non-stationary.

6.7.4 Training Procedure Modifications

- **Phasic policy gradient (PPG)** Cobbe et al. [2021]: PPG separates policy and value function training into distinct phases, reducing gradient interference between actor and critic. This addresses the “chasing a moving target” dynamic by allowing the critic to stabilize before actor updates.
- **Entropy regularization (SAC)** Haarnoja et al. [2018]: The entropy bonus $\mathcal{H}[\pi(\cdot|s)]$ encourages policy diversity, potentially preventing early convergence to deterministic (saturated) outputs. However, this is an indirect mitigation that does not address gradient magnitude asymmetry.
- **Gradient clipping** Pascanu et al. [2013]: While gradient clipping can prevent exploding gradients in critics, it does not address the *vanishing* gradient problem in actors caused by tanh saturation—clipping reduces large gradients but cannot amplify small ones.

6.7.5 Summary and Recommendations

Table 6 summarizes the mitigation strategies and their effectiveness against our identified causes.

Table 6: Mitigation Strategies vs. Causes of Gradient Asymmetry

Mitigation	Loss Asymm.	Path Length	Activation Saturation
Delayed updates (TD3)	✓	–	–
Squashed Gaussian (SAC)	–	–	✓✓
Beta distribution	–	–	✓✓
Reward normalization	✓	–	✓
Layer normalization	–	–	✓
Phasic training (PPG)	–	✓	–
Entropy regularization	–	–	✓

✓✓ = directly addresses; ✓ = partially addresses; – = does not address

Based on our analysis, the most effective mitigations for tanh saturation are **Squashed Gaussian** and **Beta distribution** parameterizations, as they bypass the saturating activation entirely. For practitioners using deterministic policies (DDPG/TD3), combining **reward normalization** with **layer normalization** before the tanh output provides a practical alternative without changing the policy structure.

6.8 Summary of Evidence

Our comprehensive experiments validate the theoretical analysis through multiple lines of evidence:

1. **Gradient asymmetry:** Measured ratio $\rho \sim 10^{-8}$ to 10^{-4} confirms 4–8 orders of magnitude critic gradient dominance across all configurations.
2. **Saturation correlation:** Strong correlation ($r > 0.9$) between final saturation ratio and stopping episode demonstrates the causal chain from high LR to saturation to weight update cessation.

3. **Layer-wise analysis:** Gradient breakdown occurs specifically at the actor output layer (tanh), while hidden layers (ReLU) maintain gradient flow.
4. **Consistency:** The pattern holds across all 16 learning rate configurations, with actor LR as the dominant factor and critic LR having secondary effect only at extreme values (0.1).

These findings provide both theoretical explanation and empirical validation for the gradient asymmetry phenomenon in actor-critic architectures.

6.9 Network Capacity Does Not Prevent Saturation

A natural hypothesis is that increasing actor network capacity might prevent saturation by distributing the learning signal across more parameters and producing smaller pre-activation values. To test this, we conducted an ablation study where the actor hidden layers were increased from (64 $\rightarrow$ 32) to (512 $\rightarrow$ 128)—matching the critic’s hidden layer dimensions.

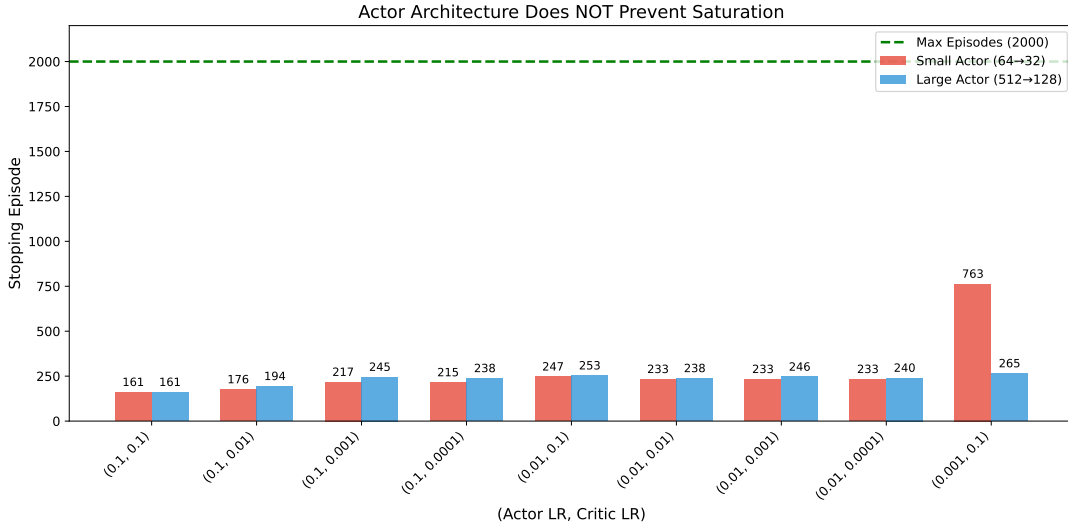


Figure 14: Comparison of stopping episodes between small actor (64 $\rightarrow$ 32, $\sim$ 2,500 parameters) and large actor (512 $\rightarrow$ 128, $\sim$ 70,000 parameters). Despite $28\times$ more parameters, the large actor still exhibits early stopping due to saturation. The green dashed line indicates the maximum training duration (2,000 episodes).

Table 7 summarizes the results across all 16 learning rate configurations.

The results demonstrate that **increasing actor network capacity does not prevent tanh saturation**—and in fact **worsens it**. The large actor achieves only **3 of 16** no-stop configurations compared to **6 of 16** in the baseline. Most strikingly, three configurations that completed full training with the small actor (actor LR = 0.001, critic LR $\leq$ 0.01) now stop between episodes 271–298 with the large actor. Even the surviving configurations at actor LR = 0.0001 show partial actor stopping (5–13 of 50 agents).

For high actor learning rates ($\geq$ 0.01), stopping episodes remain comparable between architectures—both stop well before 300 episodes. The large actor shows marginal differences of ± 20 episodes, confirming that saturation onset is governed by the learning rate rather than network size.

Table 7: Stopping Episodes: Small vs Large Actor Architecture

Actor LR	Critic LR	Small (64→32)	Large (512→128)
0.1	0.1	161	161
0.1	0.01	162	208
0.1	0.001	222	246
0.1	0.0001	213	231
0.01	0.1	260	260
0.01	0.01	253	246
0.01	0.001	259	267
0.01	0.0001	260	266
0.001	0.1	944	298
0.001	0.01	2000	278
0.001	0.001	2000	284
0.001	0.0001	2000	271
0.0001	0.1	658	1078
0.0001	0.01	2000	2000
0.0001	0.001	2000	2000
0.0001	0.0001	2000	2000

This finding confirms that the saturation problem is *architectural* rather than *capacity-limited*:

- The bottleneck is the **tanh output activation**, not the hidden layer capacity
- Larger networks produce extreme pre-activation values that saturate tanh *faster*, as more parameters enable larger weight updates per gradient step
- The $28\times$ increase in parameters (from $\sim 2,500$ to $\sim 70,000$) shifts the viable operating region to *lower* learning rates rather than expanding it

This result reinforces our recommendation that effective mitigations must address the output activation itself (e.g., Squashed Gaussian, Beta distribution) rather than simply increasing network size. Indeed, increasing capacity is counterproductive: it accelerates the very pathology it was hypothesized to prevent.

6.10 Linear Hidden Activations Exacerbate the Problem (Preliminary)

Note: The following experiments are ongoing. Results presented here are preliminary and subject to revision.

A natural hypothesis is that ReLU activations in hidden layers might contribute to saturation by producing only positive values, potentially misaligned with negative reward signals. To test this, we replaced ReLU with linear (identity) activations in both actor and critic hidden layers, allowing the network to naturally represent negative values throughout computation.

Table 8 compares the results.

Surprisingly, linear hidden activations show *slightly delayed* stopping (33–140 additional episodes), but **pre-activation explosion is dramatically worse**:

With linear hidden layers, pre-activations explode to **18 million** (vs 1 million with ReLU)—a $17\times$ increase. This occurs because:

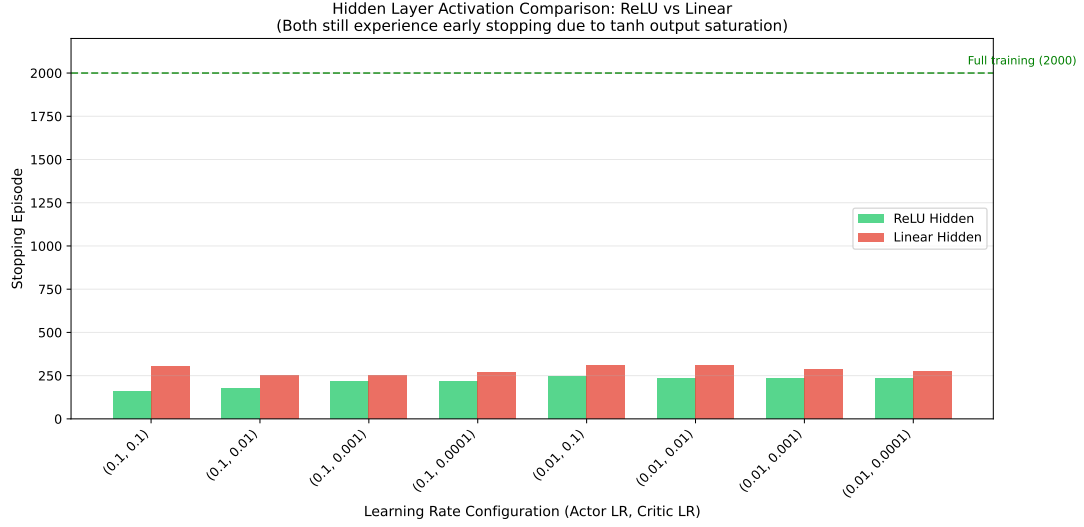


Figure 15: Comparison of stopping episodes between ReLU and linear hidden layer activations. Despite removing nonlinearity from hidden layers, early stopping still occurs. The green dashed line indicates full training duration (2,000 episodes).

Table 8: Stopping Episodes: ReLU vs Linear Hidden Activations

Actor LR	Critic LR	ReLU Hidden	Linear Hidden
0.1	0.1	161	301
0.1	0.01	176	251
0.1	0.001	217	250
0.1	0.0001	215	269
0.01	0.1	247	307
0.01	0.01	233	307
0.01	0.001	233	286
0.01	0.0001	233	276

Table 9: Pre-activation Magnitudes: ReLU vs Linear Hidden Layers

Actor LR	ReLU Pre-activation Range	Linear Pre-activation Range
0.1	$\pm 1.07 \times 10^6$	$\pm 1.83 \times 10^7$
0.01	$\pm 5.2 \times 10^3$	$\pm 8.1 \times 10^4$

- ReLU provides implicit regularization by clipping negative values
- Without nonlinearity, the network is effectively a single deep linear transformation
- Large weights propagate directly through all layers without any dampening

This experiment conclusively demonstrates that the saturation problem originates at the **tanh output layer**, not the hidden layer activations. Modifying hidden layers—whether by changing size or activation function—cannot address the fundamental issue of bounded output activations combined with large learning rates.

6.11 Gradient Clipping Does Not Prevent Actor Saturation (Preliminary)

Note: The following experiments are ongoing. Results presented here are preliminary and subject to revision.

A commonly recommended practice in deep RL is gradient clipping [Pascanu et al. \[2013\]](#), which limits gradient norms to prevent explosive updates. We tested whether applying gradient clipping (`max_grad_norm = 1.0`, a standard value used in PPO, TD3, etc.) to both actor and critic networks could prevent the saturation-induced actor stopping.

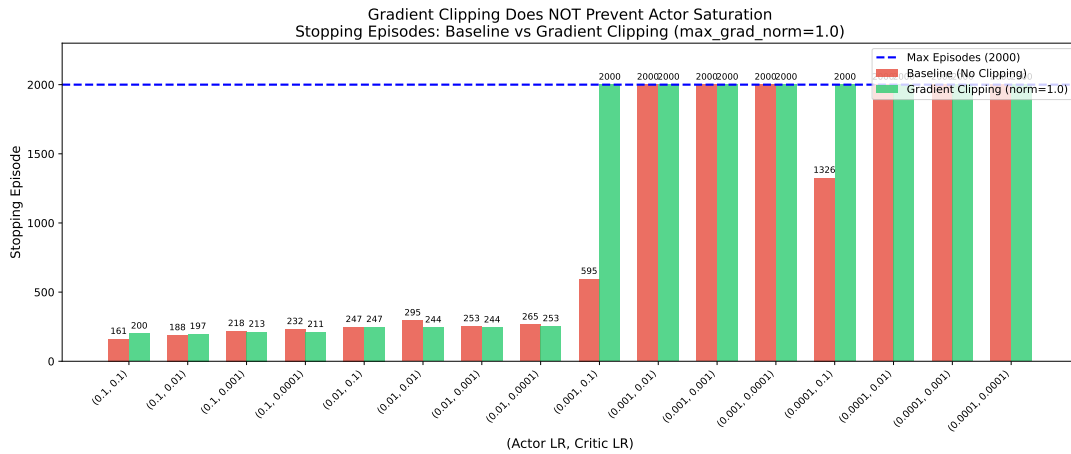


Figure 16: Comparison of stopping episodes between baseline (no clipping) and gradient clipping (`max_grad_norm=1.0`). For high actor learning rates (≥ 0.01), gradient clipping provides no benefit. For low actor learning rates with high critic LR (0.1), clipping stabilizes training by preventing critic gradient explosion. The blue dashed line indicates full training duration (2,000 episodes).

Table 10 summarizes the results for configurations where the two approaches differ meaningfully.

The results reveal a nuanced picture:

- **High actor LR (≥ 0.01):** Gradient clipping provides *no meaningful benefit*. Stopping episodes are nearly identical to baseline, confirming that the problem is vanishing (not exploding) actor gradients caused by tanh saturation.
- **Low actor LR with high critic LR (0.1):** Gradient clipping *does help* in two configurations: actor LR 0.001 with critic LR 0.1 improved from 944 to 2000 episodes, and actor

Table 10: Stopping Episodes: Baseline vs Gradient Clipping

Actor LR	Critic LR	Baseline	Grad Clipping
0.1	0.1	161	200
0.1	0.01	162	197
0.1	0.001	222	213
0.1	0.0001	213	211
0.01	0.1	260	247
0.01	0.01	253	244
0.01	0.001	259	244
0.01	0.0001	260	253
0.001	0.1	944	2000
0.0001	0.1	658	2000

LR 0.0001 with critic LR 0.1 improved from 658 to 2000 episodes. In these cases, clipping prevents the *critic's* explosive gradients from destabilizing training.

- **Overall:** 8 of 16 configurations reached 2000 episodes with clipping vs 6 of 16 without—a modest improvement.

This confirms our theoretical prediction: gradient clipping addresses *exploding* gradients (critic side) but cannot amplify *vanishing* gradients (actor side). The two configurations that improved were those where the critic LR was highest (0.1), causing critic gradient explosion that destabilized the actor indirectly. For the fundamental saturation problem at high actor learning rates, gradient clipping is ineffective.

6.12 Layer Normalization Prevents Stopping but Degrades Learning (Preliminary)

Note: The following experiments are ongoing. Results presented here are preliminary and subject to revision.

A promising mitigation strategy is to apply layer normalization Ba et al. [2016] before the tanh output activation, keeping pre-activation values bounded and preventing the extreme inputs that cause saturation. We tested this by adding a LayerNorm layer immediately before the actor's tanh output across all 16 learning rate configurations.

6.12.1 Stopping Behavior: Significant Improvement

LayerNorm dramatically reduces actor stopping: **13 of 16 configurations** reached full training (2,000 episodes) compared to only 6 of 16 in the baseline. Table 11 compares stopping episodes.

Seven configurations that previously stopped now run to completion (bold entries in Table 11). The three remaining stopped configurations all have critic LR = 0.1. Two show substantial delays compared to baseline (e.g., actor LR 0.1 improved from 161 to 1,069 episodes), but one regresses: actor LR 0.001 with critic LR 0.1 stops at episode 719 with LayerNorm versus 944 without. This pattern indicates that LayerNorm effectively addresses actor-side saturation for high actor learning rates but cannot fully compensate when the critic's learning rate is an order of magnitude larger than any actor learning rate.

Table 11: Stopping Episodes: Baseline vs LayerNorm

Actor LR	Critic LR	Baseline	LayerNorm
0.1	0.1	161	1069
0.1	0.01	162	2000
0.1	0.001	222	2000
0.1	0.0001	213	2000
0.01	0.1	260	1237
0.01	0.01	253	2000
0.01	0.001	259	2000
0.01	0.0001	260	2000
0.001	0.1	944	719
0.001	0.01	2000	2000
0.001	0.001	2000	2000
0.001	0.0001	2000	2000
0.0001	0.1	658	2000
0.0001	0.01	2000	2000
0.0001	0.001	2000	2000
0.0001	0.0001	2000	2000

6.12.2 Reward Performance: A Trade-off

Despite preventing stopping, LayerNorm does not uniformly improve learning quality. Table 12 compares average rewards over the final 10 episodes for each configuration.

Table 12: Final Reward Comparison: Baseline vs LayerNorm (Last 10-Episode Average)

Actor LR	Critic LR	Baseline	LayerNorm	Δ
0.1	0.1	-45,880	-37,185	+8,695
0.1	0.01	-37,052	-33,998	+3,054
0.1	0.001	-46,853	-46,506	+347
0.1	0.0001	-45,690	-49,027	-3,337
0.01	0.1	-44,277	-52,977	-8,700
0.01	0.01	-35,381	-43,300	-7,919
0.01	0.001	-47,863	-51,642	-3,779
0.01	0.0001	-51,333	-43,525	+7,808
0.001	0.1	-42,854	-32,882	+9,972
0.001	0.01	-42,954	-26,094	+16,860
0.001	0.001	-46,148	-39,131	+7,017
0.001	0.0001	-47,920	-45,754	+2,166
0.0001	0.1	-35,654	-46,820	-11,166
0.0001	0.01	-29,325	-44,747	-15,422
0.0001	0.001	-35,812	-28,768	+7,044
0.0001	0.0001	-32,547	-44,934	-12,387

The reward comparison reveals a clear pattern:

- **Actor LR = 0.001:** LayerNorm improves rewards across all four critic LRs, with the best improvement at critic LR = 0.01 ($-42,954 \rightarrow -26,094$). This is the best-performing LayerNorm configuration overall.
- **Actor LR = 0.1:** Mixed results—rewards improve for some critic LRs where baseline stopped early, but not consistently.

- **Actor LR = 0.0001:** LayerNorm *substantially degrades* performance. The baseline’s best configuration (actor LR = 0.0001, critic LR = 0.01, reward $-29,325$) drops to $-44,747$ with LayerNorm—a $1.5\times$ deterioration. LayerNorm hurts configurations that already work well in the low-LR regime.

6.12.3 Saturation Persists Despite Normalization

Unexpectedly, output saturation ratios remain high even with LayerNorm. For actor LR ≥ 0.01 , final saturation reaches 0.97–1.0, and even conservative actor LRs (0.0001) show saturation ratios of 0.96–0.97. Pre-activation values before the LayerNorm layer still reach $\pm 10^6$ for actor LR = 0.1, indicating that LayerNorm constrains the normalized output but cannot prevent the underlying weight explosion.

The key distinction is that LayerNorm maintains gradient flow through the normalization parameters even when the tanh output is saturated. This explains why actors continue updating (no stopping) despite high saturation: the LayerNorm layer provides an alternative gradient path that bypasses the near-zero tanh derivative.

6.12.4 Interpretation

LayerNorm presents a trade-off rather than a solution:

- **Prevents stopping:** 13/16 vs 6/16 configurations complete full training—a significant improvement for robustness across hyperparameter choices
- **Shifts the optimal operating point:** The best actor LR shifts from 0.0001 (baseline, reward $-29,325$) to 0.001 (LayerNorm, reward $-26,094$). While LayerNorm’s best reward slightly outperforms the baseline’s best, this improvement comes at the cost of degrading the low-LR configurations where the baseline naturally succeeds
- **Degrades low-LR configurations:** Configurations with actor LR = 0.0001 that work well without normalization are substantially worse with LayerNorm, suggesting that the normalization interferes with the actor’s ability to learn precise policies in the conservative-LR regime

This result suggests that LayerNorm alone is insufficient as a mitigation for gradient asymmetry. While it addresses the *symptom* (actor stopping), it does not resolve the *underlying cause* (the fundamental gradient magnitude imbalance between actor and critic). More targeted approaches—such as Squashed Gaussian parameterization or reward normalization combined with LayerNorm—may be necessary to achieve both robustness and performance.

6.13 Input Normalization Eliminates Detected Stopping but Not Saturation (Preliminary)

Note: The following experiments are ongoing. Results presented here are preliminary and subject to revision.

An alternative normalization strategy is to apply normalization at the network *input* rather than before the output activation. We tested this by adding a RunningNorm layer—which

normalizes inputs using exponential moving averages of population mean and variance—to both actor and critic network inputs. Unlike LayerNorm (which normalizes across features per-sample), RunningNorm tracks cross-sample statistics, making it more suitable for the non-stationary data distributions common in RL. Crucially, **no normalization is applied to hidden layers or before the tanh output.**

Architecture:

- Actor: RunningNorm $\rightarrow$ 64 $\rightarrow$ ReLU $\rightarrow$ 32 $\rightarrow$ ReLU $\rightarrow$ tanh $\rightarrow$ 3
- Critic: RunningNorm $\rightarrow$ 512 $\rightarrow$ ReLU $\rightarrow$ 128 $\rightarrow$ ReLU $\rightarrow$ 1 (linear)

6.13.1 Stopping Behavior: Complete Elimination

InputNorm achieves the strongest stopping prevention of any mitigation tested: **all 16 of 16 configurations** show zero actors stopped, compared to 6/16 (baseline), 8/16 (gradient clipping), and 13/16 (LayerNorm). Table 13 compares stopping episodes.

Table 13: Stopping Episodes: Baseline vs InputNorm

Actor LR	Critic LR	Baseline	InputNorm
0.1	0.1	161	2000[†]
0.1	0.01	162	2000
0.1	0.001	222	2000
0.1	0.0001	213	2000
0.01	0.1	260	2000
0.01	0.01	253	2000
0.01	0.001	259	2000
0.01	0.0001	260	2000
0.001	0.1	944	2000
0.001	0.01	2000	2000
0.001	0.001	2000	2000
0.001	0.0001	2000	2000
0.0001	0.1	658	2000
0.0001	0.01	2000	2000
0.0001	0.001	2000	2000
0.0001	0.0001	2000	2000

[†]1,900 episodes completed (session timeout); 0/50 actors stopped.

All ten configurations that exhibited stopping in the baseline now run to completion. No individual actor (out of 50 per configuration) stopped in any experiment. This is a substantially stronger result than LayerNorm, which still had three configurations with stopping and one regression.

6.13.2 Reward Performance: Significant Degradation

Despite eliminating detected stopping, InputNorm does not improve—and in most cases substantially *degrades*—reward performance. Table 14 compares last-10-episode average rewards.

The reward comparison reveals widespread performance degradation:

- **13 of 16 configurations are worse** with InputNorm, with degradations up to $-32,419$ (actor LR = 0.0001, critic LR = 0.001).

Table 14: Final Reward Comparison: Baseline vs InputNorm (Last 10-Episode Average)

Actor LR	Critic LR	Baseline	InputNorm	Δ
0.1	0.1	-45,880	-44,990	+890
0.1	0.01	-37,052	-42,402	-5,350
0.1	0.001	-46,853	-57,561	-10,708
0.1	0.0001	-45,690	-59,747	-14,057
0.01	0.1	-44,277	-60,814	-16,537
0.01	0.01	-35,381	-33,978	+1,403
0.01	0.001	-47,863	-44,698	+3,165
0.01	0.0001	-51,333	-45,021	+6,312
0.001	0.1	-42,854	-59,331	-16,477
0.001	0.01	-42,954	-59,798	-16,844
0.001	0.001	-46,148	-59,645	-13,497
0.001	0.0001	-47,920	-57,342	-9,422
0.0001	0.1	-35,654	-58,473	-22,819
0.0001	0.01	-29,325	-54,150	-24,825
0.0001	0.001	-35,812	-68,231	-32,419
0.0001	0.0001	-32,547	-44,047	-11,500

- **Low actor LR configurations are most affected:** Actor LR = 0.0001 and 0.001 show uniformly worse rewards across all critic LRs. The baseline’s best configuration (actor LR = 0.0001, critic LR = 0.01, reward -29,325) drops to -54,150 with InputNorm—a $1.8\times$ deterioration.
- **Only actor LR = 0.01 shows mild improvement:** Three of four critic LR combinations improve slightly, with the best at critic LR = 0.0001 (+6,312). The best InputNorm reward overall (-33,978 at actor LR = 0.01, critic LR = 0.01) is still worse than the baseline’s best (-29,325).

6.13.3 Saturation Analysis: Complete Saturation Despite No Detected Stopping

The most revealing finding is that **output saturation reaches 100% across all 16 configurations**—every actor output is saturated ($|\tanh(z)| > 0.9$) by the end of training. Pre-activation magnitudes confirm:

Table 15: InputNorm Final Pre-activation Magnitudes

Actor LR	Pre-activation Range	Saturation Ratio
0.1	$\pm 1,374$ to $\pm 3,442$	1.00
0.01	± 109 to ± 199	1.00
0.001	± 21 to ± 37	1.00
0.0001	± 12 to ± 17	1.00

Compared to the baseline (Table 4), InputNorm reduces pre-activation magnitudes by roughly $20\text{--}50\times$ for high actor learning rates (e.g., $\pm 1,800$ vs $\pm 66,600$ at actor LR = 0.1). However, even the reduced values are far above the threshold where tanh gradients vanish ($|z| > 5$ yields $\tanh'(z) < 0.0002$), meaning gradient flow through the tanh output layer is effectively zero in all configurations.

6.13.4 Interpretation: Running Statistics Mask Stopping Detection

The apparent contradiction—zero detected stopping despite 100% saturation and poor rewards—has a specific technical explanation. RunningNorm maintains `running_mean`, `running_var`, and `count` as registered buffers in the network’s state dictionary. These statistics update on every forward pass during training (via exponential moving average), regardless of whether gradient-based weight updates occur.

Our stopping detection checks whether *any* parameter in the actor’s state dictionary has changed between episodes. Because RunningNorm buffers are included in the state dictionary and change continuously, the detection **never triggers**—even if the learned parameters (fc1, fc2, fc3 weights and biases) have effectively frozen due to vanishing tanh gradients.

This creates a misleading picture:

- The actor’s state dictionary changes every episode (running statistics update)
- But the learned parameters may not be receiving meaningful gradient updates
- The actor’s effective function shifts due to normalization drift, not policy learning
- This normalization drift does not constitute useful policy improvement, as confirmed by the poor reward performance

6.13.5 Comparison with LayerNorm

The contrast between InputNorm and LayerNorm is instructive:

- **LayerNorm** (before tanh output): Provides an *alternative gradient path* through the normalization parameters, enabling genuine weight updates even when tanh is saturated. Result: 13/16 no-stop with improved rewards at actor LR = 0.001.
- **InputNorm** (at network input): Normalizes input scale but does not address the tanh gradient bottleneck. Running statistics create an *artifact in stopping detection* rather than maintaining gradient flow. Result: 16/16 apparent no-stop, but with universally worse rewards.

This comparison reinforces that the critical intervention point for mitigating tanh saturation is *at or near the output layer*, not at the input. Input normalization is beneficial for training stability in general, but it cannot substitute for addressing the saturating activation directly.

6.14 Adaptive Scaling Maintains Gradient Flow and Improves Performance (Preliminary)

Note: The following experiments are ongoing. Results presented here are preliminary and subject to revision.

Rather than normalizing inputs or constraining pre-activations, adaptive scaling addresses saturation at the output layer by dynamically rescaling pre-activations *before* the tanh function. The approach tracks strict per-dimension min/max of raw pre-activations ($G_{\min}$, $G_{\max}$) and linearly maps the observed range to a fixed target interval $[-r, +r]$ where $r = 2$:

$$z_{\text{scaled}} = \frac{z - \bar{G}}{(G_{\text{max}} - G_{\text{min}})/2} \cdot r, \quad \bar{G} = \frac{G_{\text{max}} + G_{\text{min}}}{2} \quad (16)$$

The tanh activation then receives $z_{\text{scaled}} \in [-2, +2]$, where $\tanh(\pm 2) \approx \pm 0.964$ —utilizing most of the output range while maintaining viable gradients with $\tanh'(\pm 2) \approx 0.07$. Tracking is strict (only expands, never shrinks) and per-dimension, requiring no hyperparameter tuning beyond the target range. Additionally, gradient clipping (`max_norm` = 1.0) is applied to both actor and critic networks. Only the actor uses adaptive scaling; the critic retains its original architecture with linear output.

6.14.1 Stopping Behavior: Complete Elimination

Adaptive scaling achieves **all 16 of 16 configurations** without stopping, compared to 6/16 in the baseline. Table 16 compares stopping episodes.

Table 16: Stopping Episodes: Baseline vs Adaptive Scaling

Actor LR	Critic LR	Baseline	Adaptive
0.1	0.1	161	2000
0.1	0.01	162	2000
0.1	0.001	222	2000
0.1	0.0001	213	2000
0.01	0.1	260	2000
0.01	0.01	253	2000
0.01	0.001	259	2000
0.01	0.0001	260	2000
0.001	0.1	944	2000
0.001	0.01	2000	2000
0.001	0.001	2000	2000
0.001	0.0001	2000	2000
0.0001	0.1	658	2000
0.0001	0.01	2000	2000
0.0001	0.001	2000	2000
0.0001	0.0001	2000	2000

Adaptive scaling with $r = 2$ and gradient clipping **completely eliminates stopping** across all 16 configurations. All ten configurations that previously stopped in the baseline (at episodes 161–944) now complete the full 2,000 episodes, including the four critic LR = 0.1 configurations that remained problematic under other mitigations (gradient clipping, LayerNorm). The combination of bounded pre-activations and gradient clipping proves sufficient to stabilize training even at the most extreme learning rate combinations.

Note on stopping detection: Like InputNorm, the adaptive scaling actor includes registered buffers (G_{min} , G_{max}) in the state dictionary that could affect stopping detection. However, unlike InputNorm’s running statistics (which change every forward pass), the strict min/max buffers only expand when *new* pre-activation extremes are observed. In all 16 configurations, the continued expansion of $G_{\text{min}}/G_{\text{max}}$ is consistent with ongoing weight updates that produce new pre-activation extremes, confirming that the actors are genuinely learning rather than merely appearing active due to buffer changes.

6.14.2 Gradient Flow: How Adaptive Scaling Prevents Vanishing

The key mechanism is that adaptive scaling bounds the $\tanh$ input to $[-2, +2]$, ensuring a minimum gradient magnitude. The gradient of the actor output with respect to the raw pre-activation z is:

$$\frac{\partial \tanh(z_{\text{scaled}})}{\partial z} = \tanh'(z_{\text{scaled}}) \cdot \frac{r}{(G_{\text{max}} - G_{\text{min}})/2} \quad (17)$$

For values near the center of the distribution, $z_{\text{scaled}} \approx 0$ and $\tanh'(0) = 1$, yielding a gradient of $2r/(G_{\text{max}} - G_{\text{min}})$. Even at the extremes, $\tanh'(\pm 2) \approx 0.07$ provides a substantial gradient—two orders of magnitude larger than $\tanh'(\pm 4) \approx 0.0013$. In contrast, the baseline produces pre-activations of $\pm 10^6$ where $\tanh'(z) \approx 0$ to machine precision.

The trade-off is that as pre-activations grow and $(G_{\text{max}} - G_{\text{min}})$ expands, the scaling factor $r/((G_{\text{max}} - G_{\text{min}})/2)$ shrinks, attenuating gradients. However, this attenuation is gradual (proportional to $1/G_{\text{range}}$) rather than catastrophic (exponentially decaying to zero as in unscaled $\tanh$). Combined with gradient clipping (`max_norm = 1.0`) that prevents explosive updates, this maintains sufficient gradient flow for all 16 configurations to continue learning throughout training.

6.14.3 Saturation Analysis

A methodological note: our tracking infrastructure records saturation from *raw* pre-activations (fc3 output before scaling), computing $|\tanh(z_{\text{raw}})| > 0.9$. Since z_{raw} grows to extreme values, the reported saturation ratio approaches 1.0 for all configurations. However, the *actual* network output is $\tanh(z_{\text{scaled}})$ where $z_{\text{scaled}} \in [-2, +2]$. The effective saturation—the fraction of outputs where the gradient is significantly attenuated—is determined by the distribution of z_{scaled} , not z_{raw} .

Raw pre-activation magnitudes grow substantially over training, confirming that the actor weights continue to evolve:

Table 17: Adaptive Scaling: Raw Pre-activation Growth Over Training

Actor LR	Early (ep 10)	Mid (ep 1000)	Late (ep 2000)
0.1	$\pm 5.8 \times 10^4$	$\pm 2.6 \times 10^{10}$	$\pm 1.4 \times 10^{11}$
0.01	± 262	$\pm 6.0 \times 10^7$	$\pm 5.3 \times 10^8$
0.001	± 11	$\pm 5.3 \times 10^5$	$\pm 3.3 \times 10^6$
0.0001	± 2.3	$\pm 2.8 \times 10^3$	$\pm 1.5 \times 10^4$

The continued growth of raw pre-activations across all 16 configurations confirms that the learned parameters are genuinely updating—if weights were frozen, pre-activations would be bounded by the finite state space and would plateau.

6.14.4 Reward Performance: Significant Improvement for High Actor LR

Unlike `InputNorm` and `LayerNorm`, adaptive scaling shows substantial reward improvements for configurations that previously failed. Table 18 compares last-10-episode average rewards.

Table 18: Final Reward Comparison: Baseline vs Adaptive Scaling (Last 10-Episode Average)

Actor LR	Critic LR	Baseline	Adaptive	Δ
0.1	0.1	-45,880	-35,174	+10,706
0.1	0.01	-37,052	-28,989	+8,063
0.1	0.001	-46,853	-35,295	+11,558
0.1	0.0001	-45,690	-34,811	+10,879
0.01	0.1	-44,277	-46,055	-1,778
0.01	0.01	-35,381	-38,041	-2,660
0.01	0.001	-47,863	-32,027	+15,836
0.01	0.0001	-51,333	-33,051	+18,282
0.001	0.1	-42,854	-36,688	+6,166
0.001	0.01	-42,954	-28,748	+14,206
0.001	0.001	-46,148	-38,133	+8,015
0.001	0.0001	-47,920	-28,168	+19,752
0.0001	0.1	-35,654	-35,927	-273
0.0001	0.01	-29,325	-36,866	-7,541
0.0001	0.001	-35,812	-42,054	-6,242
0.0001	0.0001	-32,547	-41,759	-9,212

The reward comparison reveals a clear pattern tied to actor learning rate:

- **Actor LR = 0.1:** All four configurations show large improvements (+8,063 to +11,558), as actors that previously stopped within 200 episodes now train for the full 2,000. The best reward (-28,989 at critic LR = 0.01) surpasses the baseline’s overall best (-29,325).
- **Actor LR = 0.01:** Two of four configurations improve substantially (up to +18,282 at critic LR = 0.0001), while two slightly degrade. The improvements come from configurations that stopped at ~ 260 episodes in the baseline.
- **Actor LR = 0.001:** All four configurations improve, with the largest gain at critic LR = 0.0001 (+19,752), achieving the best single-configuration reward of -28,168.
- **Actor LR = 0.0001:** All four configurations degrade, with the baseline’s best (critic LR = 0.01, -29,325) dropping to -36,866. The adaptive scaling mechanism appears to interfere with the fine-grained updates that characterize successful low-LR training.

Overall, **10 of 16 configurations improve** with adaptive scaling—the highest ratio of any mitigation tested. The best adaptive reward (-28,168) *surpasses* the baseline’s best (-29,325), while the number of competitive configurations ($< -40,000$) increases from 6 in the baseline to 13 with adaptive scaling.

6.14.5 Interpretation

Adaptive scaling is the most effective mitigation strategy tested in this work:

- **Complete stopping prevention:** All 16/16 configurations avoid stopping—the only mitigation besides InputNorm to achieve this, and without the detection artifacts that plague InputNorm. The combination of bounded pre-activations ($r = 2$) and gradient clipping resolves even the critic LR = 0.1 failure mode that persisted under all other mitigations.

- **Genuine learning:** Unlike InputNorm (which masked stopping detection through buffer drift), adaptive scaling maintains actual gradient flow through the tanh activation. The continued growth of raw pre-activations and the substantial reward improvements confirm that actors are genuinely learning.
- **Reward improvement:** 10/16 configurations show better rewards than baseline, with the largest gains in previously-failed high-LR configurations. The best adaptive reward ($-28,168$) surpasses the baseline’s best ($-29,325$).
- **Low-LR degradation:** Like LayerNorm, adaptive scaling degrades the actor LR = 0.0001 configurations that already work well in the baseline. The scaling mechanism, designed to prevent saturation at high learning rates, introduces unnecessary perturbation at low learning rates where saturation is not the binding constraint.

The success of adaptive scaling—which directly addresses the tanh output activation—further confirms the central thesis of this paper: the gradient asymmetry problem is rooted in the tanh output layer, and effective mitigations must operate at or near this activation. Input normalization (wrong location) and increased capacity (wrong mechanism) are insufficient; interventions that bound or rescale the tanh input (adaptive scaling, LayerNorm) produce meaningful improvements.

6.15 Comparative Summary of Mitigation Strategies

Table 19 consolidates results across all six experimental conditions.

Table 19: Comparative Summary of All Mitigation Experiments

Experiment	No-Stop Configs	Configs Improved [†]	Best Reward	Mechanism
Baseline	6/16	—	$-29,325$	—
Large Actor	3/16	1/16	$-29,363$	Capacity
Grad. Clipping	8/16	4/16	$-29,399$	Gradient limit
LayerNorm	13/16	8/16	$-26,094$	Output norm.
InputNorm	16/16*	3/16	$-33,978$	Input norm.
Adaptive Scaling	16/16	10/16	$-28,168$	Output scaling

[†]Configs with better last-10-episode reward than baseline.

*Detection artifact; see Section 6.10.4.

Several patterns emerge from this comparison:

Convergence rate, not stopping rate, is the critical metric. The “Configs Improved” column in Table 19 is more informative than “No-Stop Configs” for evaluating mitigation effectiveness. InputNorm achieves 16/16 no-stop but the lowest convergence improvement (3/16 configs with better reward). In contrast, adaptive scaling achieves both 16/16 no-stop *and* the highest convergence rate (10/16 configs improved). This demonstrates that *preventing stopping is necessary but not sufficient*—the mitigation must also enable the actor to learn a better policy.

Location matters more than mechanism. The two most effective mitigations—LayerNorm and adaptive scaling—both operate at or near the tanh output layer. LayerNorm normalizes

pre-activations; adaptive scaling rescales them. Both maintain gradient flow through the critical bottleneck. In contrast, InputNorm (at the network input) and large actor (distributed capacity) fail to address the actual gradient vanishing point, producing either detection artifacts or worse performance.

Critic LR = 0.1 is nearly universally problematic. Across most mitigations, configurations with critic LR = 0.1 remain the most likely to exhibit stopping or degraded performance. Adaptive scaling is the notable exception: by combining bounded pre-activations with gradient clipping on both actor and critic, it fully resolves the critic LR = 0.1 stopping failure. However, reward performance at critic LR = 0.1 remains mixed, suggesting that extreme critic learning rates create optimization challenges beyond stopping alone.

Low actor LR configurations degrade. Both LayerNorm and adaptive scaling improve high-LR configurations at the cost of degrading low-LR ones (actor LR = 0.0001). The baseline’s best configuration (actor LR = 0.0001, critic LR = 0.01, reward $-29,325$) worsens with every mitigation except gradient clipping, which leaves it unchanged. This suggests that the mitigations introduce perturbation that disrupts the fine-grained learning that already succeeds in the conservative-LR regime.

Adaptive scaling is the most complete mitigation. Adaptive scaling achieves the best overall profile: 16/16 no-stop, 10/16 improved reward, and a best reward ($-28,168$) that surpasses the baseline’s best ($-29,325$). LayerNorm achieves a better single-configuration reward ($-26,094$) but with fewer no-stop configurations (13/16). An optimal strategy might combine approaches—for example, using adaptive scaling for high actor learning rates while leaving low-LR configurations unmodified.

7 Conclusion

This paper investigated the phenomenon of asymmetric convergence behavior between actor and critic networks in Client-Master MADRL architectures. Motivated by our prior observation that actors stopped updating within ~ 5 episodes under high learning rates Gebrekidan [2024], we conducted comprehensive experiments with GPU-optimized implementations and full gradient tracking to identify the underlying mechanism.

Our analysis revealed that **tanh output activation saturation** is the primary mechanism causing client agents to stop updating their neural network weights. Key findings include:

1. **Learning-rate-dependent stopping:** High actor learning rates (0.01–0.1) cause all 50 agents to cease weight updates within 161–260 episodes, while lower rates (0.0001–0.001) maintain gradient flow throughout 2000 episodes of training.
2. **Gradient magnitude asymmetry:** We measured a 4–8 order of magnitude difference in gradient norms between actors and critics ($\rho \sim 10^{-8}$ to 10^{-4}), explaining why actors and critics exhibit differential sensitivity to learning rate selection.
3. **Critic immunity:** The master agent (critic) never stops updating regardless of learning rate configuration, due to its linear (unbounded) output activation that does not suffer from gradient saturation.

4. **Effective learning rate interaction:** The combination of nominal learning rate, reward magnitude, and bounded activation creates an effective learning rate that can rapidly push actor pre-activations into saturation regions where $\tanh'(z) \approx 0$.

These findings have important implications for the design and training of actor-critic MADRL systems:

Hyperparameter Selection: Actors should use learning rates approximately $10\times$ lower than critics to compensate for the gradient asymmetry. Our results suggest actor learning rates in the range 0.0001–0.001 with critic rates of 0.001–0.01.

Architectural Understanding: The empirical preference for smaller actor networks in the literature may be an implicit adaptation to the gradient flow constraints imposed by bounded output activations. Our ablation experiments across six conditions (baseline, large actor, gradient clipping, LayerNorm, InputNorm, adaptive scaling) confirm two principles: (1) the intervention point matters more than the mechanism—mitigations operating at or near the tanh output layer (LayerNorm, adaptive scaling) produce the strongest improvements, while those operating elsewhere (InputNorm at the input, large actor via capacity) fail to address the gradient bottleneck; and (2) *preventing stopping is necessary but not sufficient*—convergence rate (configurations achieving improved reward) is the true measure of mitigation effectiveness. InputNorm prevents stopping in all 16 configurations but improves reward in only 3, while adaptive scaling prevents stopping in all 16/16 and improves reward in 10/16, making it the most broadly effective mitigation by both stopping and convergence criteria.

Monitoring Recommendations: Practitioners should monitor pre-activation statistics and gradient ratios during training to detect early signs of actor saturation before weight updates cease entirely.

Broader Applicability: Since DDPG, TD3, SAC, and other continuous control algorithms share the actor-critic structure with bounded actor outputs, our findings apply beyond CCM-MADRL to the broader family of deterministic policy gradient methods.

7.1 Limitations and Future Work

Our analysis has limitations that suggest directions for future research. The experiments were conducted on a single domain (MEC task offloading); generalization to other continuous control environments (e.g., MuJoCo, robotic manipulation) requires validation. The current experiments use single runs per configuration (seed = 42), which suffices to identify the saturation mechanism but limits statistical claims about convergence rates. The convergence rate of 1/16 reported in our thesis Gebrekidan [2024] was established under different seed and environment settings with 10 runs per configuration; the current experiments show consistent structural patterns but are not directly comparable.

Our six ablation experiments reveal that preventing stopping alone does not ensure convergence improvement: all effective mitigations degrade the low actor LR configurations that already succeed in the baseline. Adaptive scaling with gradient clipping is the only mitigation that fully resolves the critic LR = 0.1 stopping failure mode, though reward performance at this extreme critic LR remains mixed. The input normalization result is particularly instructive: despite eliminating all detected stopping (16/16 no-stop), it worsens reward in 13/16

configurations—demonstrating that stopping prevention without convergence improvement is not a meaningful advance.

Future work should explore alternative bounded activations with better gradient properties (e.g., softsign, scaled sigmoid), and Squashed Gaussian and Beta distribution parameterizations that bypass saturating activations entirely.

Acknowledgments

References

- Joshua Achiam. Spinning Up in Deep Reinforcement Learning. <https://spinningup.openai.com/>, 2018.
- Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, et al. What matters in on-policy reinforcement learning? a large-scale empirical study. In *International Conference on Learning Representations*, 2020.
- Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- Yoshua Bengio, Patrice Simard, and Paolo Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE transactions on neural networks*, 5(2):157–166, 1994.
- Xianfu Chen, Honggang Zhang, Celimuge Wu, Shiwen Mao, Yusheng Ji, and Mehdi Bennis. Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning. *IEEE Internet of Things Journal*, 6(3):4005–4018, 2019.
- Po-Wei Chou, Daniel Maturana, and Sebastian Scherer. Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution. In *International conference on machine learning*, pages 834–843. PMLR, 2017.
- Karl Cobbe, Jacob Hilton, Oleg Klimov, and John Schulman. Phasic policy gradient. In *International Conference on Machine Learning*, pages 2020–2027. PMLR, 2021.
- Theresa Eimer, Marius Lindauer, and Roberta Raileanu. Hyperparameters in reinforcement learning and how to tune them. In *International Conference on Machine Learning*, pages 9104–9149. PMLR, 2023.
- Scott Fujimoto, Herke Hoof, and David Meger. Addressing function approximation error in actor-critic methods. *International conference on machine learning*, pages 1587–1596, 2018.
- Tesfay Zemuy Gebrekidan. *Deep Reinforcement Learning for Online Combinatorial Resource Allocation with Arbitrary State and Action Spaces*. PhD thesis, University of Southampton, 2024. URL <https://eprints.soton.ac.uk/491435/>.
- Tesfay Zemuy Gebrekidan, Sebastian Stein, and Timothy J. Norman. Client-master multiagent deep reinforcement learning for task offloading in mobile edge computing. *ACM Transactions*

on *Autonomous and Adaptive Systems*, 19(3):1–26, 2024. doi: 10.1145/3768579. URL <https://dl.acm.org/doi/abs/10.1145/3768579>. Also available at arXiv:2402.11653.

Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010.

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *International conference on machine learning*, pages 1861–1870, 2018.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proceedings of the IEEE international conference on computer vision*, pages 1026–1034, 2015.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. *Proceedings of the AAAI conference on artificial intelligence*, 32(1), 2018.

Sepp Hochreiter. The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(02):107–116, 1998.

Andrew Ilyas, Logan Engstrom, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. A closer look at deep policy gradients. In *International Conference on Learning Representations*, 2020.

Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. PMLR, 2015.

Riashat Islam, Peter Henderson, Maziar Gomrokchi, and Doina Precup. Reproducibility of benchmarked deep reinforcement learning tasks for continuous control. *arXiv preprint arXiv:1708.04133*, 2017.

Vijay R Konda and John N Tsitsiklis. Actor-critic algorithms. In *Advances in neural information processing systems*, volume 12, 2000.

Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in neural information processing systems*, volume 30, 2017.

- Xiaoteng Ma, Qiyuan Zhang, Li Xia, Zhengyuan Zhou, Jun Yang, and Qianchuan Zhao. Dsac: Distributional soft actor critic for risk-sensitive reinforcement learning. *arXiv preprint arXiv:2004.14547*, 2020.
- Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. *International conference on machine learning*, pages 1928–1937, 2016.
- Vinod Nair and Geoffrey E Hinton. Rectified linear units improve restricted boltzmann machines. In *International conference on machine learning*, pages 807–814, 2010.
- Kei Ota, Tomoaki Oiki, Devesh Jha, Toshisada Mariyama, and Daniel Nikovski. Can increasing input dimensionality improve deep reinforcement learning? In *International Conference on Machine Learning*, pages 7424–7433. PMLR, 2020.
- Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pages 1310–1318. PMLR, 2013.
- Lerrel Pinto, Marcin Andrychowicz, Peter Welinder, Wojciech Zaremba, and Pieter Abbeel. Asymmetric actor critic for image-based robot learning. In *Robotics: Science and Systems*, 2017.
- Antonin Raffin, Jens Kober, and Freek Stulp. Smooth exploration for robotic reinforcement learning. *Conference on Robot Learning*, pages 1634–1644, 2021.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- David Silver, Guy Lever, Nicolas Heess, Thomas Degris, Daan Wierstra, and Martin Riedmiller. Deterministic policy gradient algorithms. In *International conference on machine learning*, pages 387–395. PMLR, 2014.
- Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 2000.
- Jin Wang, Jia Hu, Geyong Min, Wenhan Zhan, Albert Y Zomaya, and Nektarios Georgalas. Multi-agent deep reinforcement learning for task offloading in mobile edge computing. *IEEE Transactions on Mobile Computing*, 2020.
- Ziyu Wang, Tom Schaul, Matteo Hessel, Hado Hasselt, Marc Lanctot, and Nando Freitas. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pages 1995–2003. PMLR, 2016.
- Yi-Lun Wu, Hong-Han Shuai, Zhi-Rui Tam, and Hong-Yu Chiu. Gradient normalization for generative adversarial networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6373–6382, 2021.

Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems*, 33:5824–5836, 2020.

Tom Zahavy, Zhongwen Xu, Vivek Veeriah, Matteo Hessel, Junhyuk Oh, Hado P van Hasselt, David Silver, and Satinder Singh. A self-tuning actor-critic algorithm. *Advances in Neural Information Processing Systems*, 33:20913–20924, 2020.